CSCI 423
Introduction to Algorithms

DR. RAAFAT ELFOULY

Jeffrey J. McConnell

Analysis of Algorithms

Second Edition

An Active Learning Approach

Chapter 8
Graph Algorithms

Chapter Outline

- Graph background and terminology
- Data structures for graphs
- Graph traversal algorithms
- Minimum spanning tree algorithms
- Shortest-path algorithm
- Biconnected components
- Partitioning sets

Prerequisites

- Before beginning this chapter, you should be able to:
  - Describe a set and set membership
  - Use two-dimensional arrays
  - Use stack and queue data structures
  - Use linked lists
  - Describe growth rates and order

Goals

- At the end of this chapter you should be able to:
  - Describe and define graph terms and concepts
  - Create data structures for graphs
  - Do breadth-first and depth-first traversals and searches
  - Find the minimum spanning tree for a connected graph
  - Find the shortest path between two nodes of a connected graph
  - Find the biconnected components of a connected graph

- What is a graph?

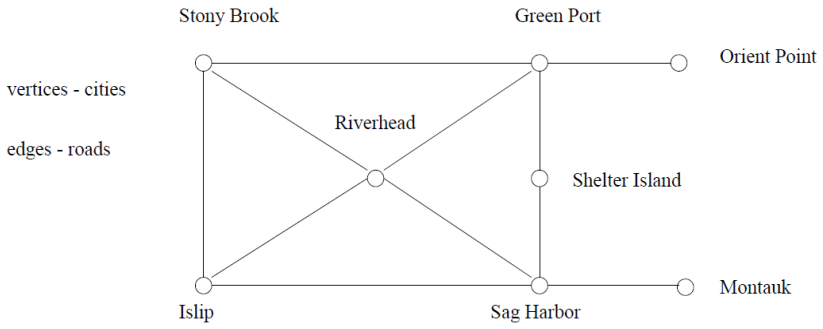
Graphs

Graphs are one of the unifying themes of computer science. A graph $G = (V, E)$ is defined by a set of *vertices* V, and a set of *edges* E consisting of ordered or unordered pairs of vertices from V.

- Why we need them?
- Examples?

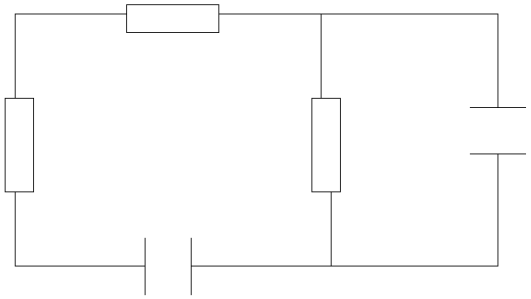
Road Networks

In modeling a road network, the vertices may represent the cities or junctions, certain pairs of which are connected by roads/edges.



Electronic Circuits

In an electronic circuit, with junctions as vertices as components as edges.



vertices: junctions

edges: components

Graph Types

- A directed graph edges' allow travel in one direction
- An undirected graph edges' allow travel in either direction

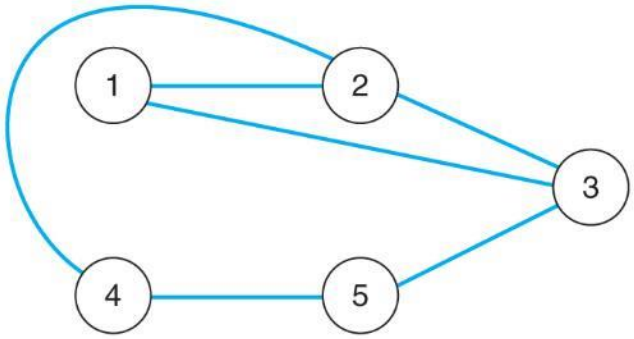


FIGURE 8.1A
The graph G = ({1, 2, 3, 4, 5}, {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}})

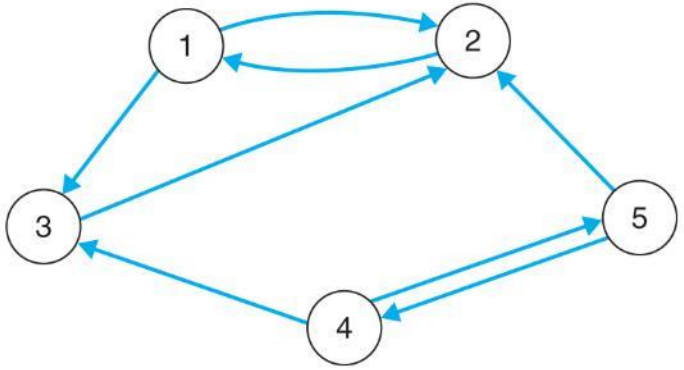


FIGURE 8.1B
The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

Graph Terminology

- A graph is an ordered pair G=(V,E) with a set of vertices or nodes and the edges that connect them
- A subgraph of a graph has a subset of the vertices and edges
- The edges indicate how we can move through the graph

Graph Terminology

- A path is a subset of E that is a series of edges between two nodes
- A graph is connected if there is at least one path between every pair of nodes
- The length of a path in a graph is the number of edges in the path

Graph Terminology

- A complete graph is one that has an edge between every pair of nodes

- A weighted graph is one where each edge has a cost for traveling between the nodes

Graph Terminology

- A cycle is a path that begins and ends at the same node
- An acyclic graph is one that has no cycles
- An acyclic, connected graph is also called an unrooted tree

Data Structures for Graphs an Adjacency Matrix

- A two-dimensional matrix or array that has one row and one column for each node in the graph
- For each edge of the graph (V_i, V_j), the location of the matrix at row *i* and column *j* is 1
- All other locations are 0

Data Structures for Graphs: Adjacency Matrix

There are two main data structures used to represent graphs. We assume the graph $G = (V, E)$ contains n vertices and m edges.

We can represent G using an $n \times n$ matrix M, where element $M[i, j]$ is, say, 1, if (i, j) is an edge of G, and 0 if it isn't. It may use excessive space for graphs with many vertices and relatively few edges, however.

Can we save space if (1) the graph is undirected? (2) if the graph is sparse?

Data Structures for Graphs
An Adjacency Matrix

- For an undirected graph, the matrix will be symmetric along the diagonal
- For a weighted graph, the adjacency matrix would have the weight for edges in the graph, zeros along the diagonal, and infinity (∞) every place else

Adjacency Matrix Example 1

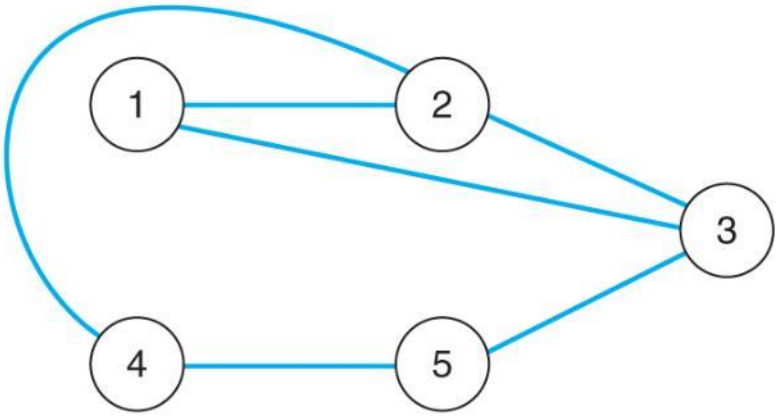


FIGURE 8.1A

The graph G = ({1, 2, 3, 4, 5}, {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}})

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

FIGURE 8.2A

The adjacency matrix for the graph in Fig. 8.1(a)

Adjacency Matrix Example 2

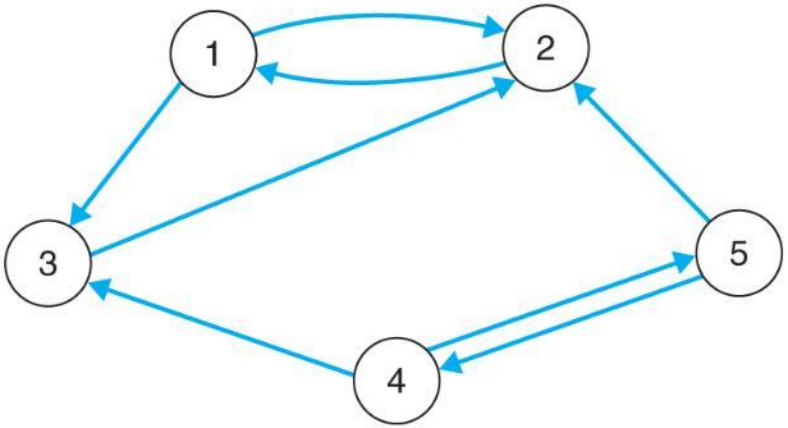


FIGURE 8.1B

The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

Adjacency Matrix Example 2

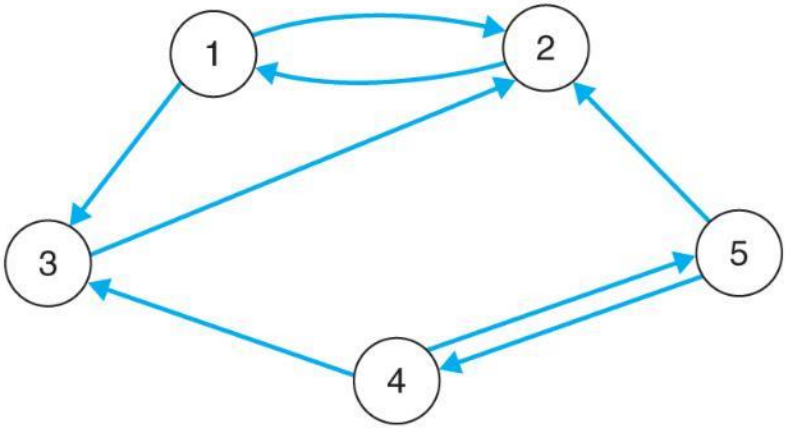


FIGURE 8.1B

The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	0	0
3	0	1	0	0	0
4	0	0	1	0	1
5	0	1	0	1	0

FIGURE 8.2B

The adjacency matrix for the digraph in Fig. 8.1(b)

Data Structures for Graphs: Adjacency Matrix

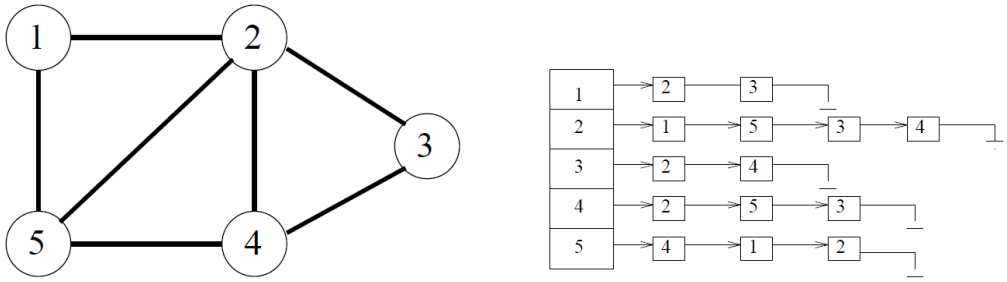
There are two main data structures used to represent graphs. We assume the graph $G = (V, E)$ contains n vertices and m edges.

We can represent G using an $n \times n$ matrix M, where element $M[i, j]$ is, say, 1, if (i, j) is an edge of G, and 0 if it isn't. It may use excessive space for graphs with many vertices and relatively few edges, however.

Can we save space if (1) the graph is undirected? (2) if the graph is sparse?

Adjacency Lists

An *adjacency list* consists of a $N \times 1$ array of pointers, where the ith element points to a linked list of the edges incident on vertex i.



To test if edge (i, j) is in the graph, we search the ith list for j, which takes $O(d_i)$, where d_i is the degree of the ith vertex. Note that d_i can be much less than n when the graph is sparse. If necessary, the two *copies* of each edge can be linked by a pointer to facilitate deletions.

Data Structures for Graphs
An Adjacency List

- A list of pointers, one for each node of the graph
- These pointers are the start of a linked list of nodes that can be reached by one edge of the graph
- For a weighted graph, this list would also include the weight for each edge

Adjacency List Example 1

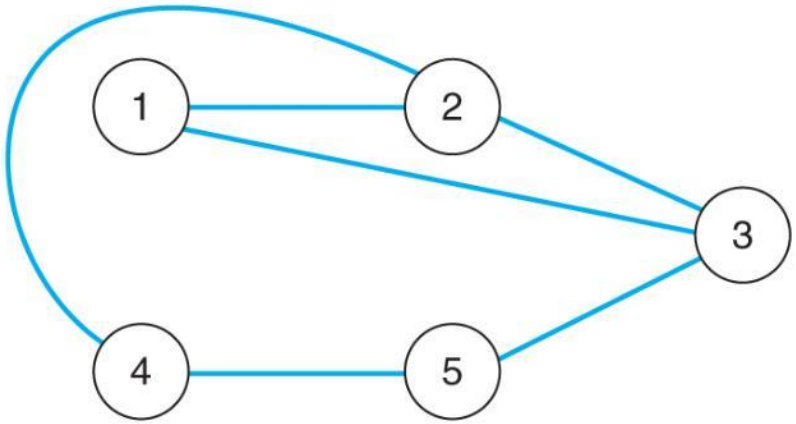


FIGURE 8.1A

The graph G = ({1, 2, 3, 4, 5}, {{1, 2}, {1, 3}, {2, 3}, {2, 4}, {3, 5}, {4, 5}})

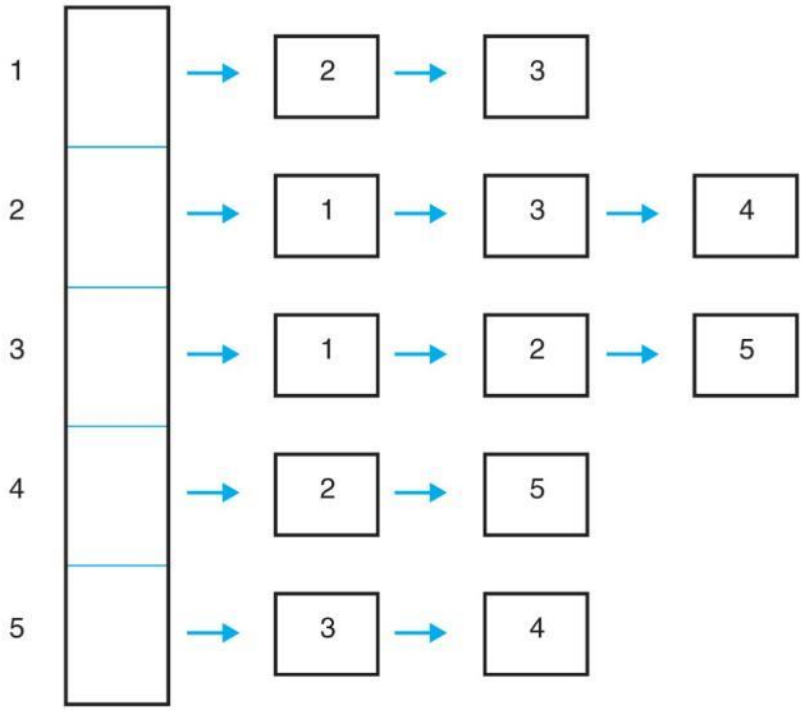


FIGURE 8.3A

The adjacency list for the graph in Fig. 8.1(a)

Adjacency List Example 2

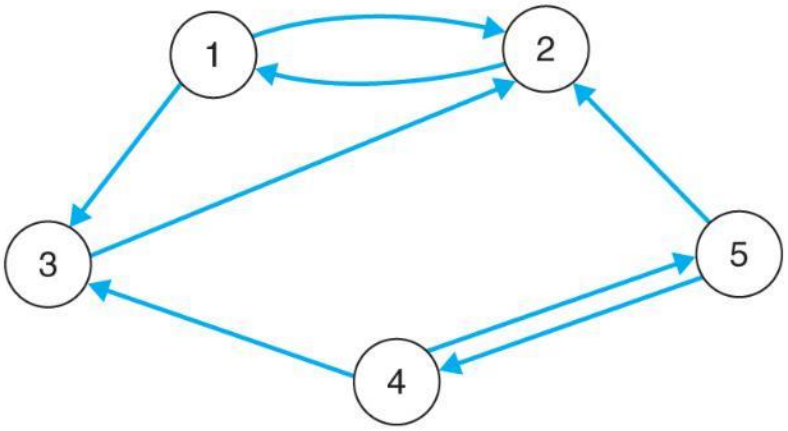


FIGURE 8.1B

The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

Adjacency List Example 2

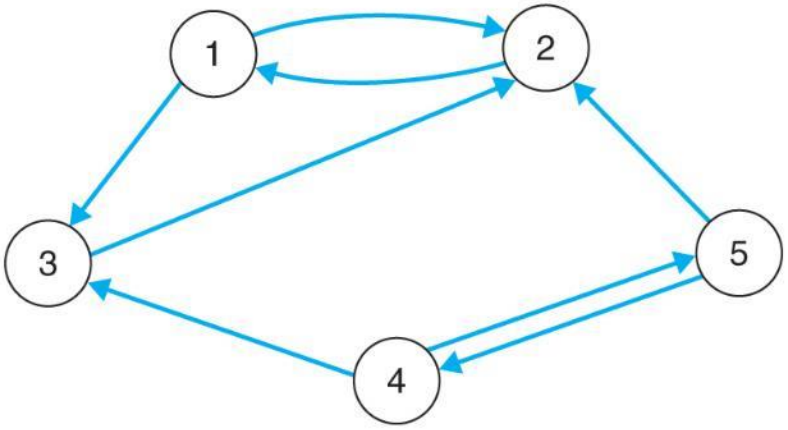


FIGURE 8.1B

The directed graph G = ({1, 2, 3, 4, 5}, {(1, 2), (1, 3), (2, 1), (3, 2), (4, 3), (4, 5), (5, 2), (5,4)})

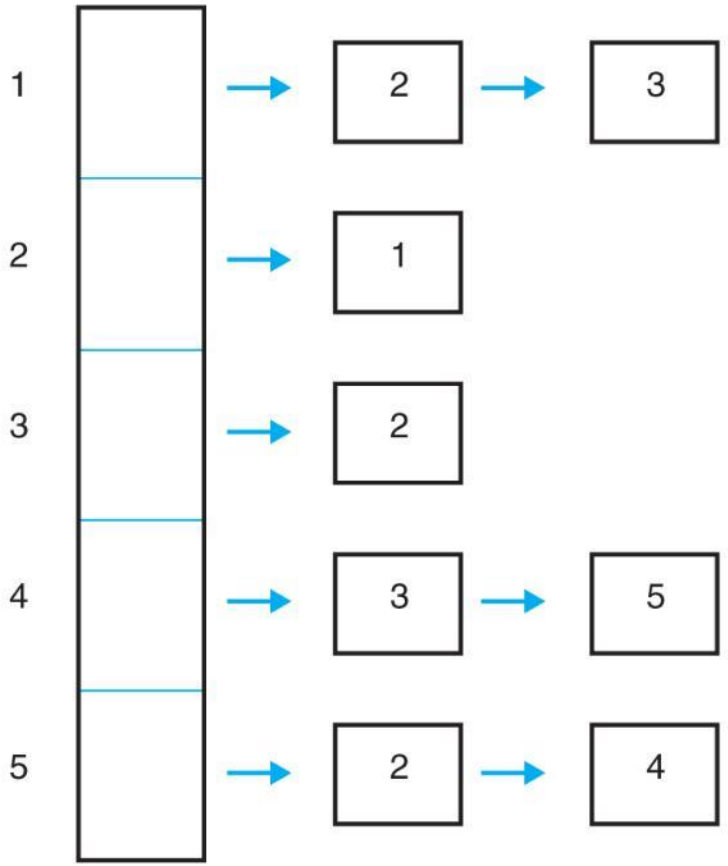


FIGURE 8.3B

The adjacency list for the graph in Fig. 8.1(b)

Tradeoffs Between Adjacency Lists and Adjacency Matrices

Comparison	Winner
Faster to test if (x, y) exists?	matrices
Faster to find vertex degree?	lists
Less memory on small graphs?	lists $(m + n)$ vs. (n^2)
Less memory on big graphs?	matrices (small win)
Edge insertion or deletion?	matrices $O(1)$
Faster to traverse the graph?	lists $m + n$ vs. n^2
Better for most problems?	lists

Both representations are very useful and have different properties, although adjacency lists are probably better for most problems.

Adjancency List Representation

```
#define MAXV 100

typedef struct {
        int y;
        int weight;
        struct edgenode *next;
} edgenode;
```

Edge Representation

```
typedef struct {
        edgenode *edges[MAXV+1];
        int degree[MAXV+1];
        int nvertices;
        int nedges;
        bool directed;
} graph;
```

The degree field counts the number of meaningful entries for the given vertex. An undirected edge (x, y) appears twice in any adjacency-based graph structure, once as y in x's list, and once as x in y's list.

Initializing a Graph

```
initialize_graph(graph *g, bool directed)
{
        int i;

        g -> nvertices = 0;
        g -> nedges = 0;
        g -> directed = directed;

        for (i=1; i<=MAXV; i++) g-> degree[i] = 0;
        for (i=1; i<=MAXV; i++) g-> edges[i] = NULL;
}
```

Reading a Graph

A typical graph format consists of an initial line featuring the number of vertices and edges in the graph, followed by a listing of the edges at one vertex pair per line.

```
read_graph(graph *g, bool directed)
{       int i;
        int m;
        int x, y;

        initialize_graph(g, directed);

        scanf("%d %d",&(g- >nvertices),&m);

        for (i=1; i<=m; i++) {
                scanf("%d %d",&x,&y);
                insert_edge(g,x,y,directed);
        }
}
```

Inserting an Edge

```
insert_edge(graph *g, int x, int y, bool directed)
{
        edgenode *p;

        p = malloc(sizeof(edgenode));

        p->weight = NULL;
        p->y = y;
        p->next = g->edges[x];

        g->edges[x] = p;

        g->degree[x] ++;

        if (directed == FALSE)
                insert_edge(g,y,x,TRUE);
        else
                g->nedges ++;
}
```

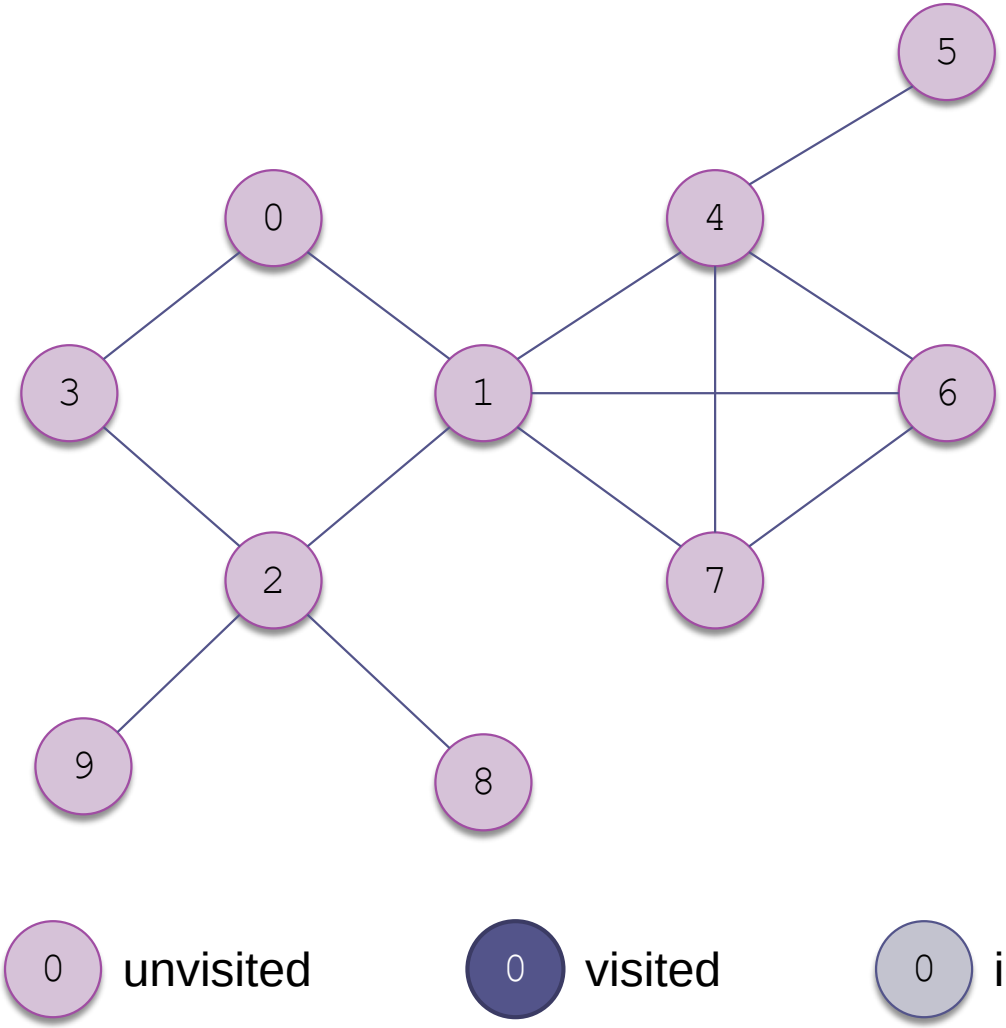
Traversals of Graphs

- Most graph algorithms involve visiting each vertex in a systematic order
- As with trees, there are different ways to do this
- The two most common traversal algorithms are the *breadth-first search* and the *depth-first search*

Breadth-First Search

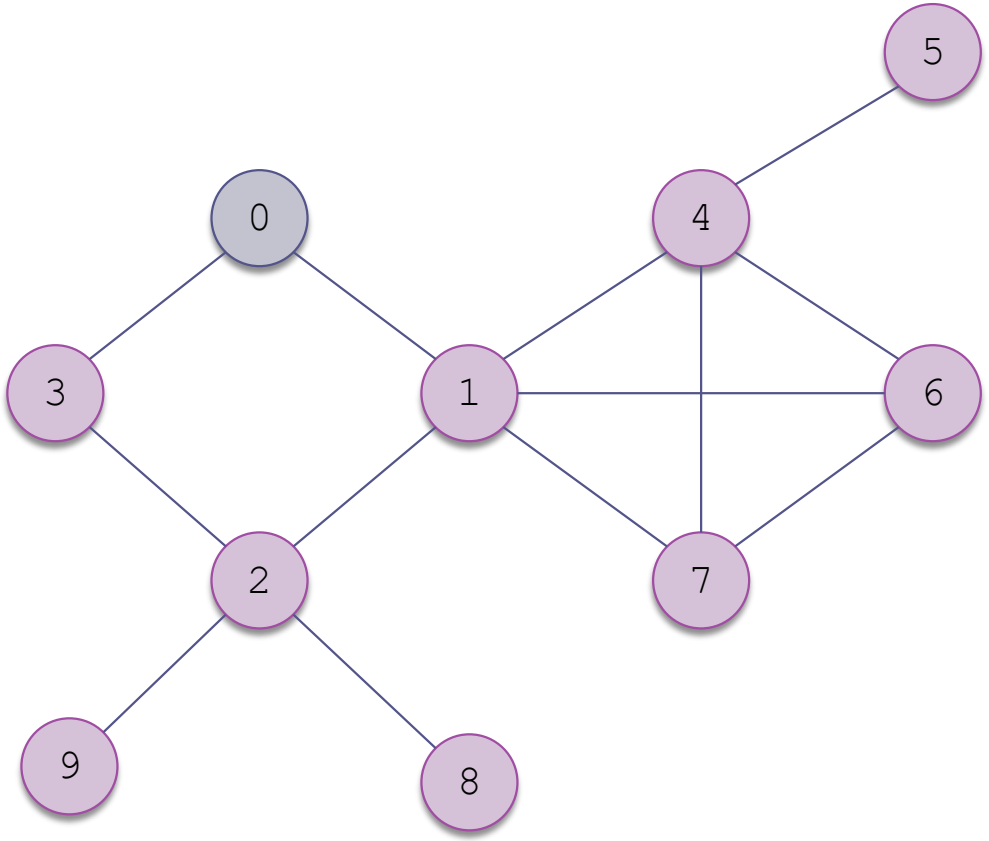
- In a breadth-first search,
  - visit the start node first,
  - then all nodes that are adjacent to it,
  - then all nodes that can be reached by a path from the start node containing two edges,
  - then all nodes that can be reached by a path from the start node containing three edges,
  - and so on
- We must visit all nodes for which the shortest path from the start node is length *k* before we visit any node for which the shortest path from the start node is length *k+1*
- There is no special start vertex– we arbitrarily choose the vertex with label 0

Example of a Breadth-First Search



Example of a Breadth-First Search (cont.)

Identify the start node



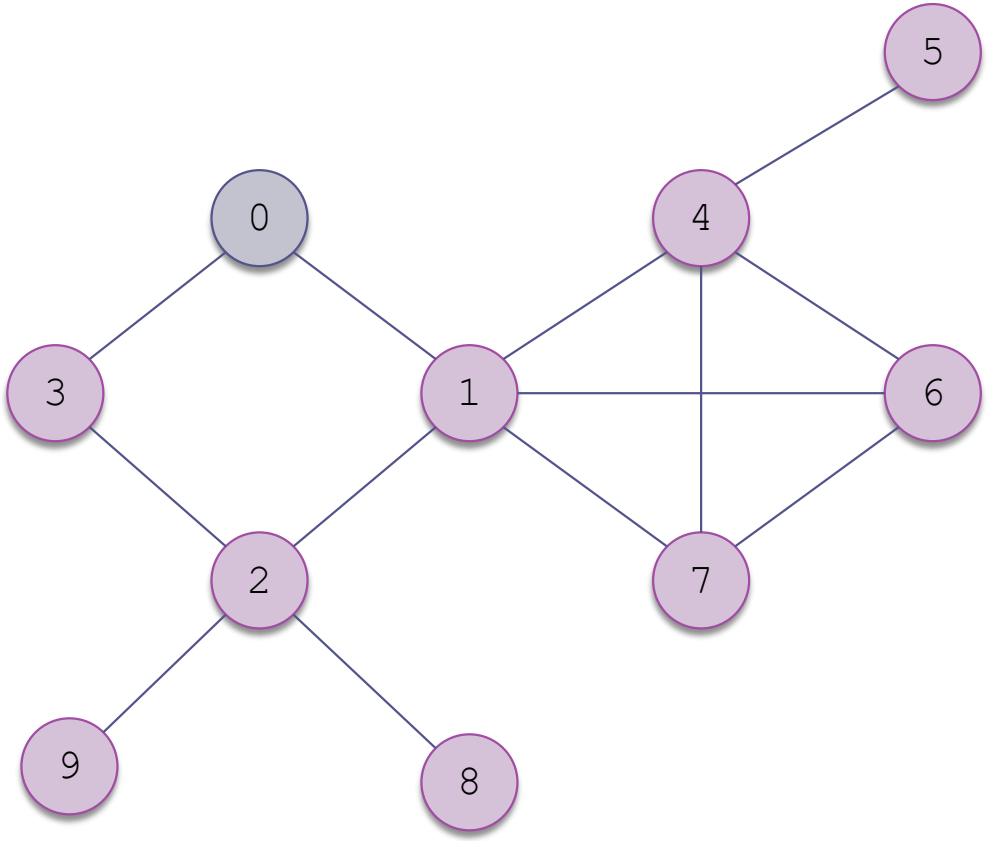
0 unvisited

0 visited

0 identified

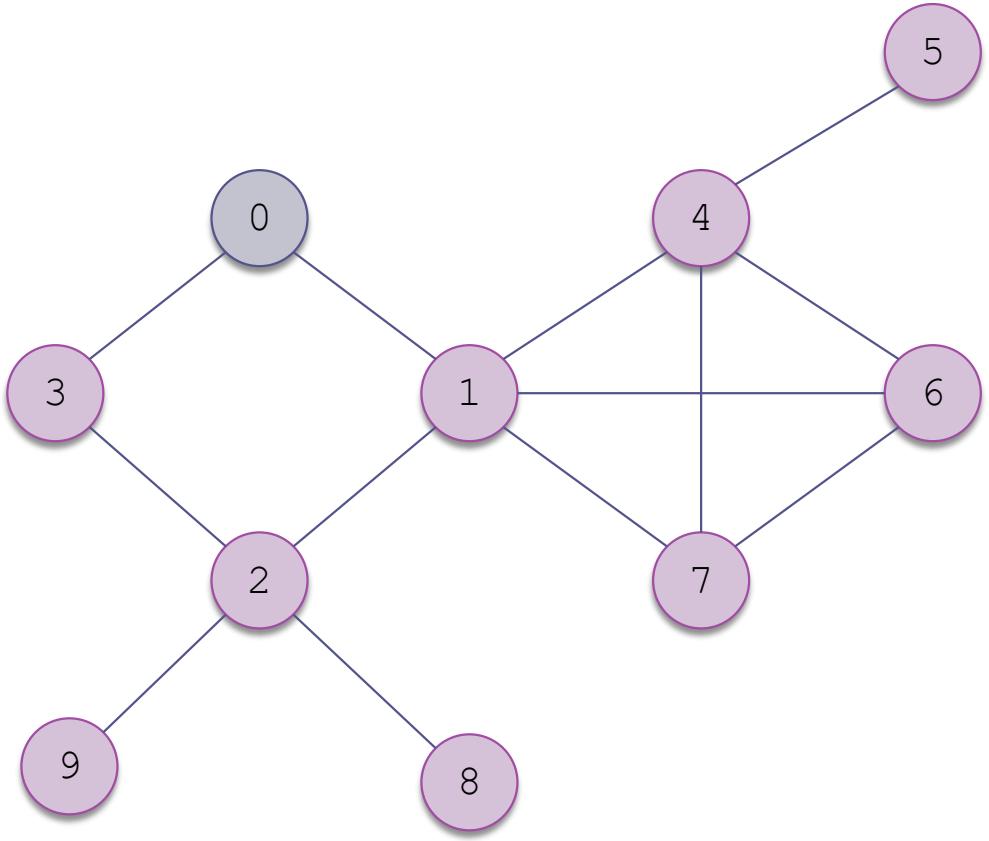
Example of a Breadth-First Search (cont.)

While visiting it, we can identify its adjacent nodes



Example of a Breadth-First Search (cont.)

We identify its adjacent nodes and add them to a queue of identified nodes



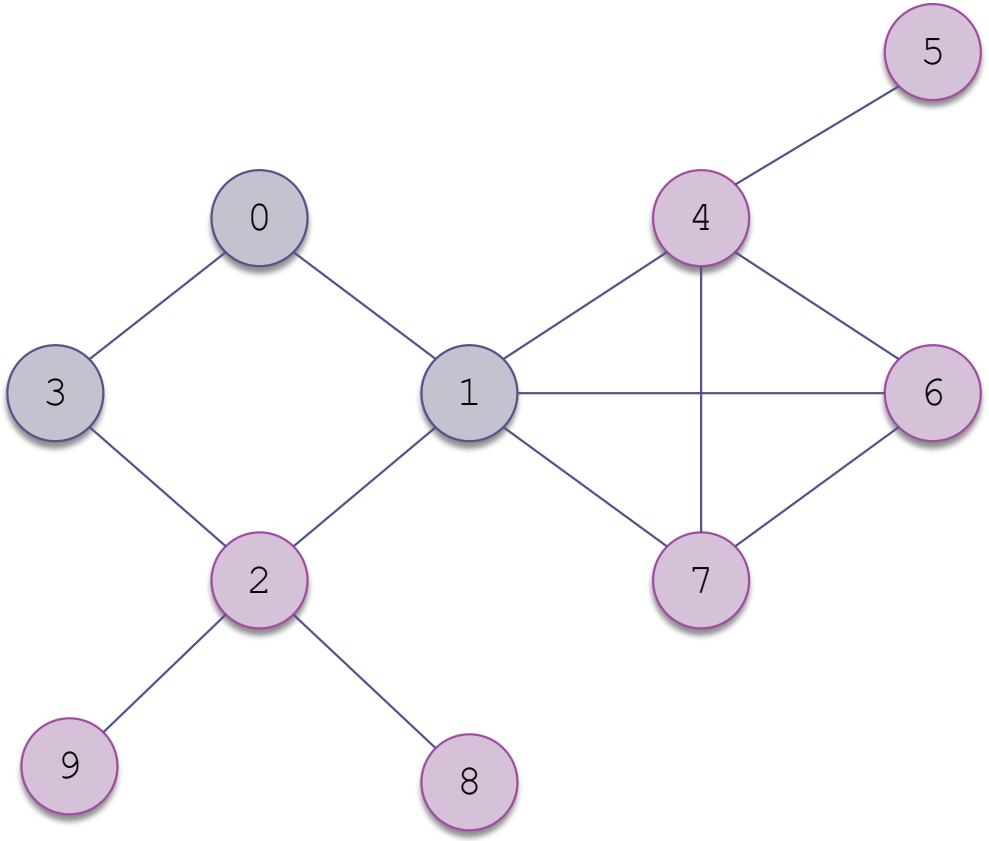
Visit sequence:
0

Example of a Breadth-First Search (cont.)

We identify its adjacent nodes and add them to a queue of identified nodes

Queue:
1, 3

Visit sequence:
0

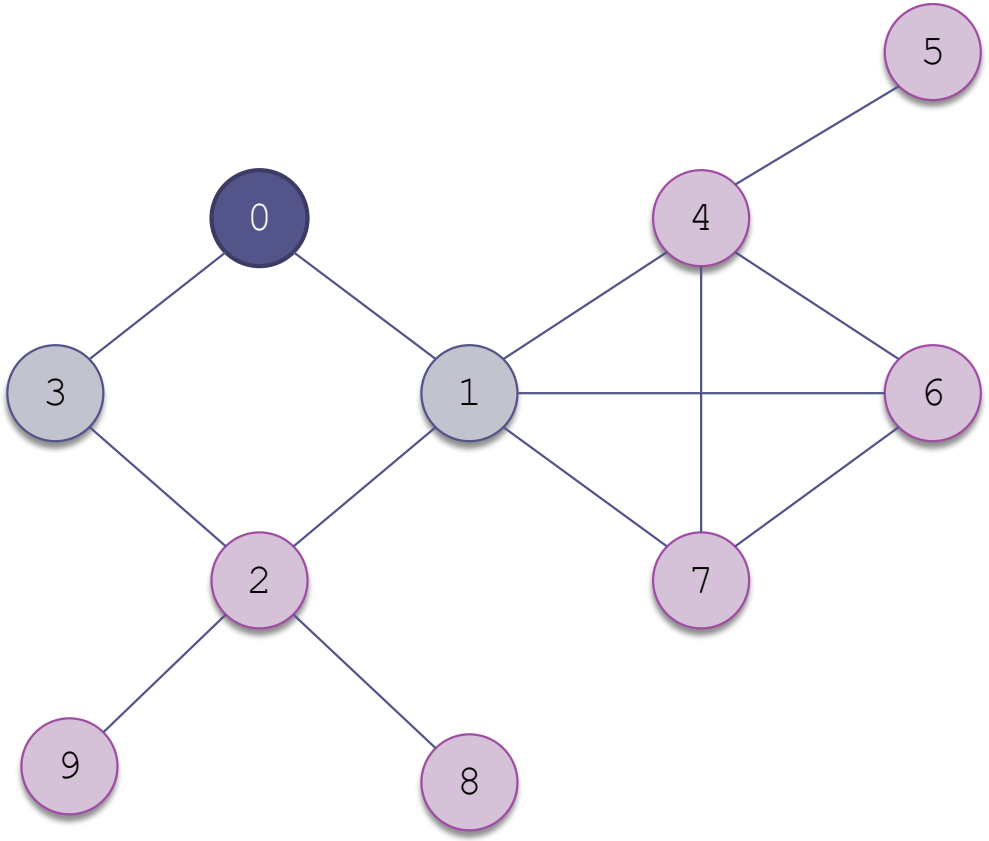


Example of a Breadth-First Search (cont.)

We color the node as visited

Queue:
1, 3

Visit sequence:
0

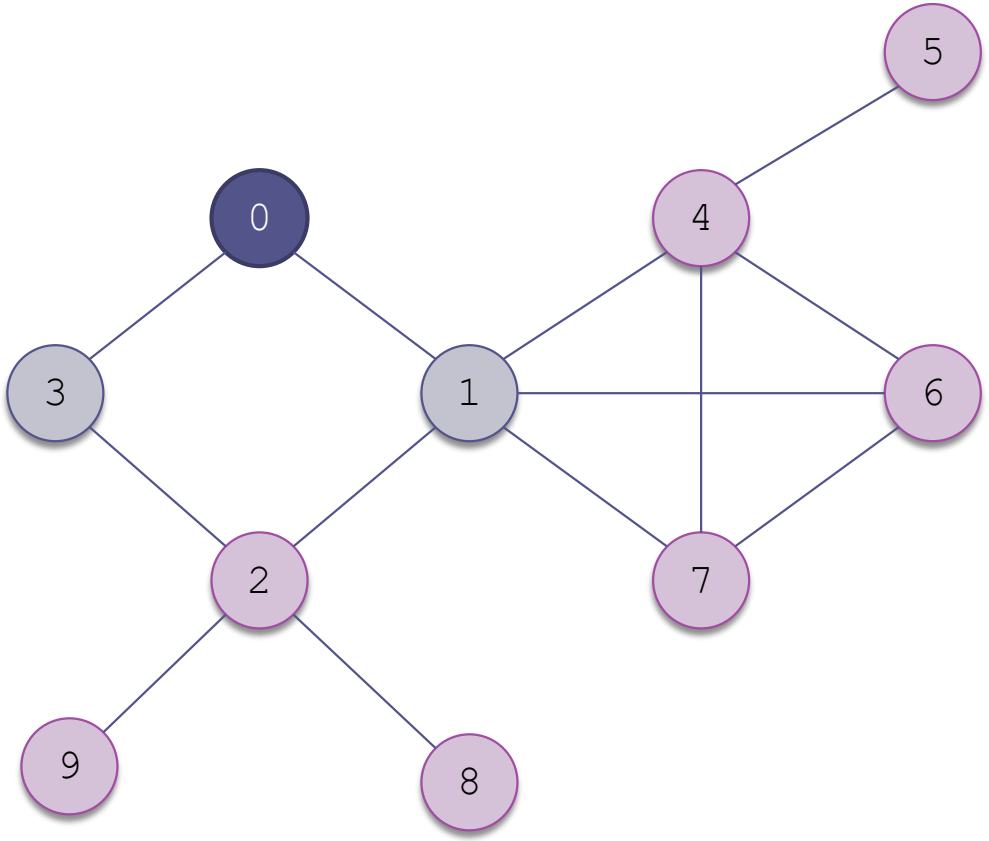


Example of a Breadth-First Search (cont.)

The queue determines which nodes to visit next

Queue:
1, 3

Visit sequence:
0



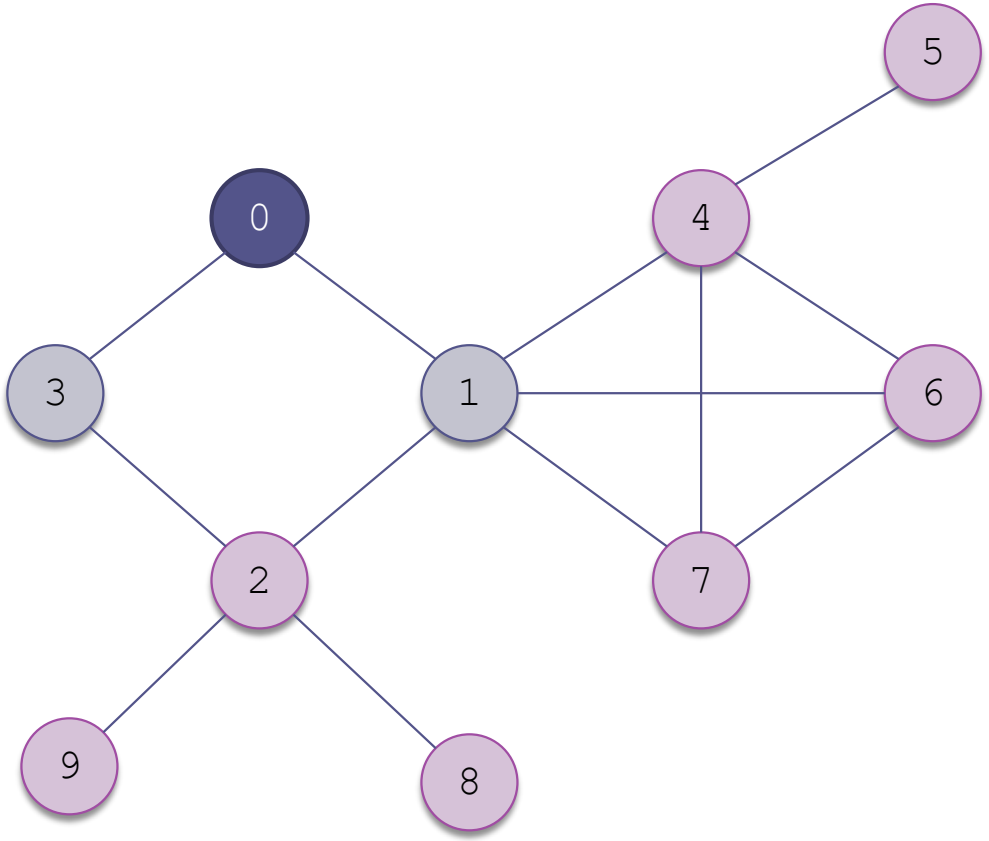
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

Visit the first node in the queue, 1

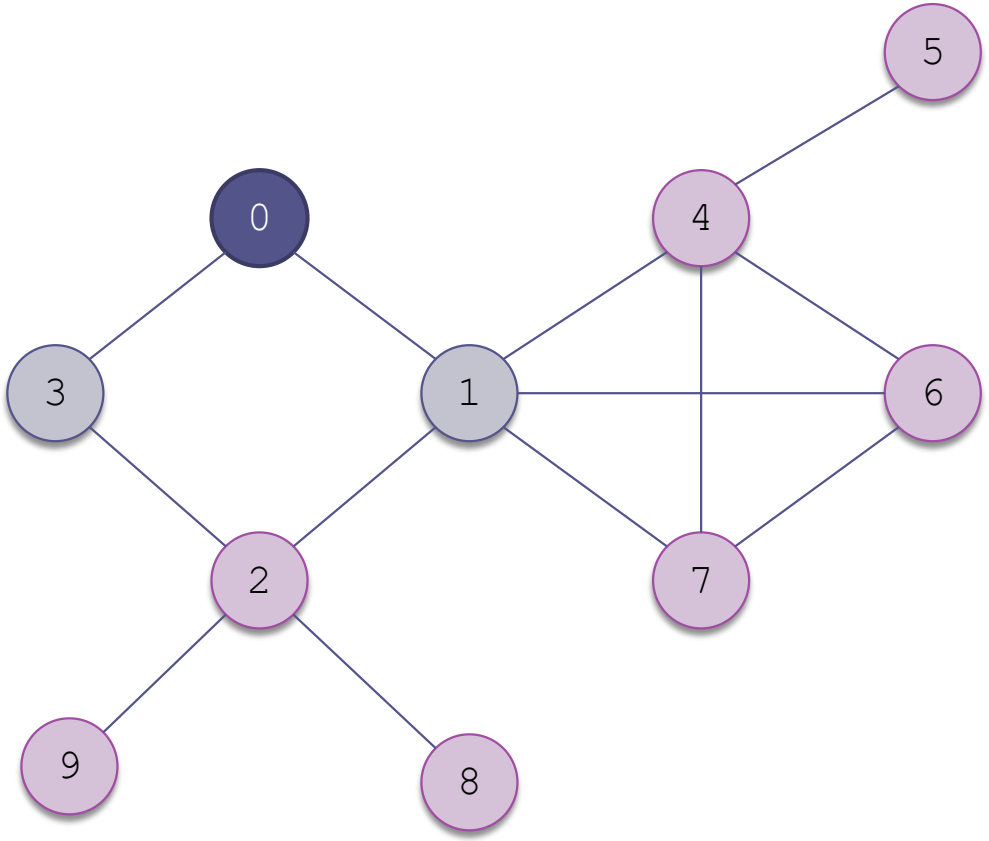
Queue:
1, 3

Visit sequence:
0



Example of a Breadth-First Search (cont.)

Visit the first node in the queue, 1



Queue:
3

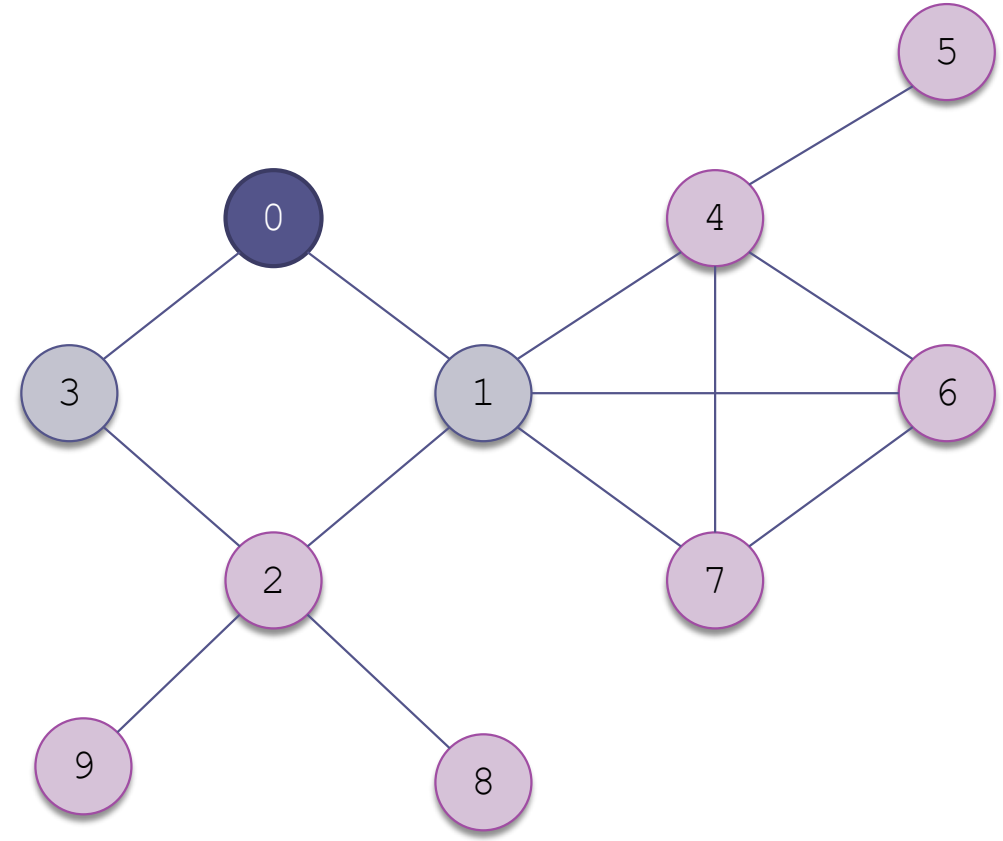
Visit sequence:
0, 1

Example of a Breadth-First Search (cont.)

Select all its adjacent nodes that have not been visited or identified

Queue:
3

Visit sequence:
0, 1

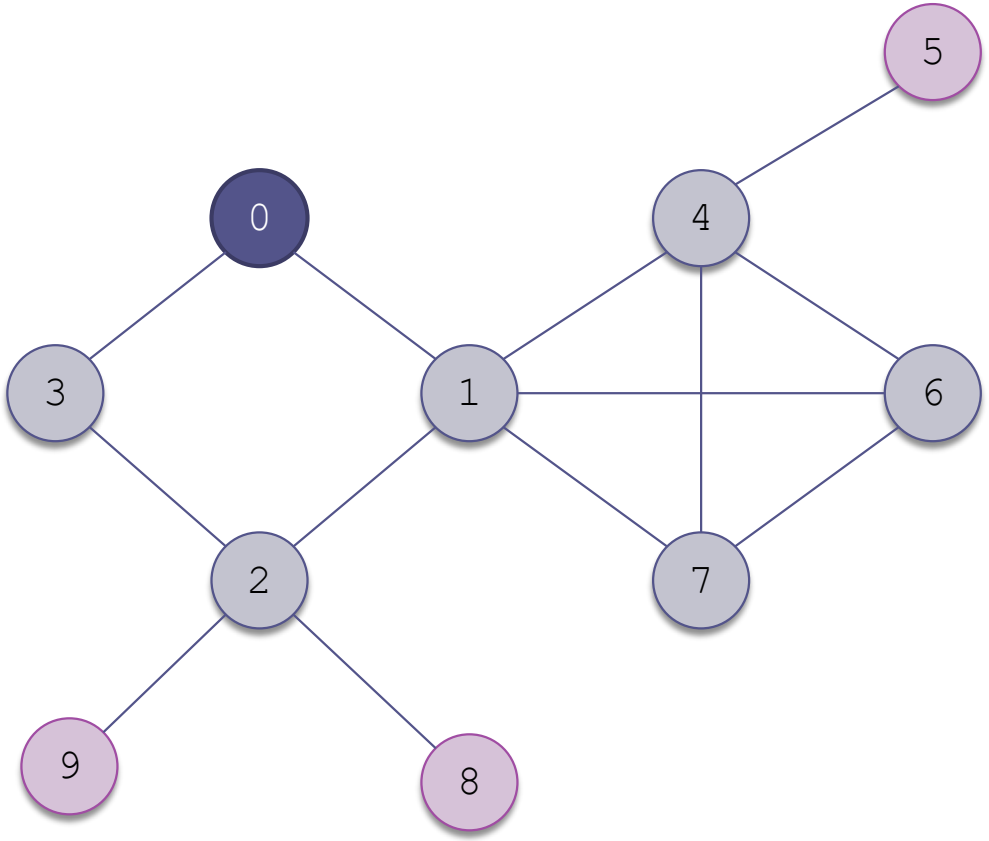


Example of a Breadth-First Search (cont.)

Select all its adjacent nodes that have not been visited or identified

Queue:
3, 2, 4, 6, 7

Visit sequence:
0, 1

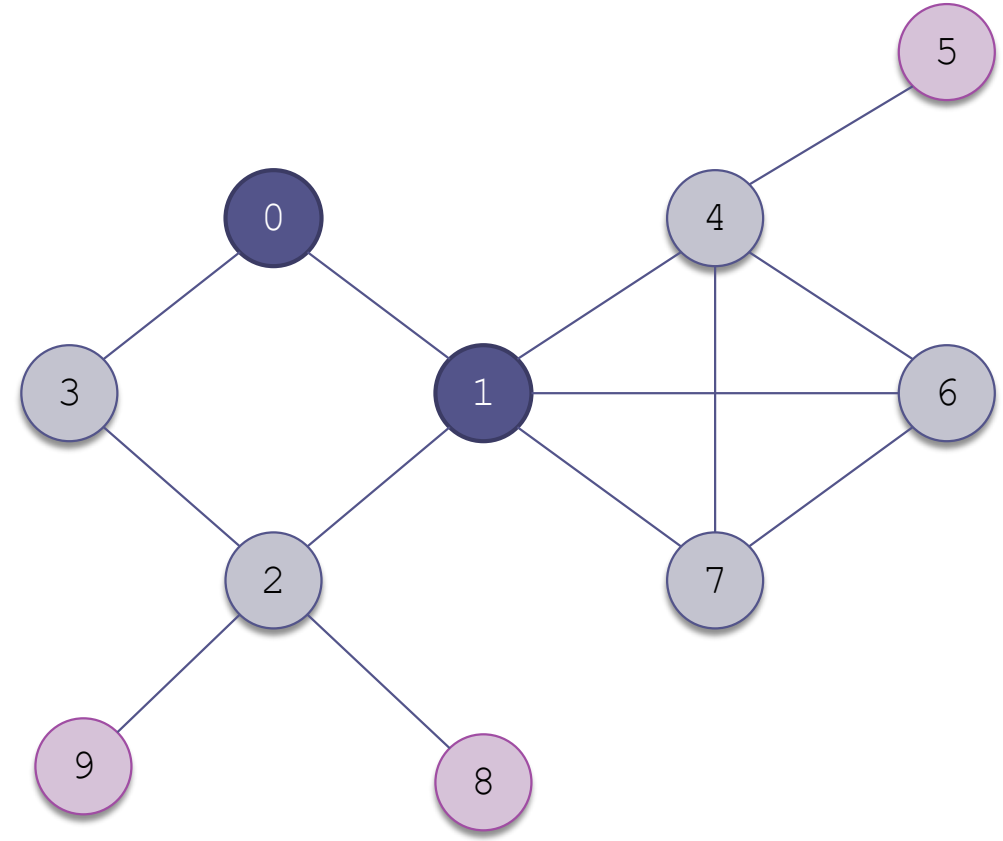


Example of a Breadth-First Search (cont.)

Now that we are done with 1, we color it as visited

Queue:
3, 2, 4, 6, 7

Visit sequence:
0, 1



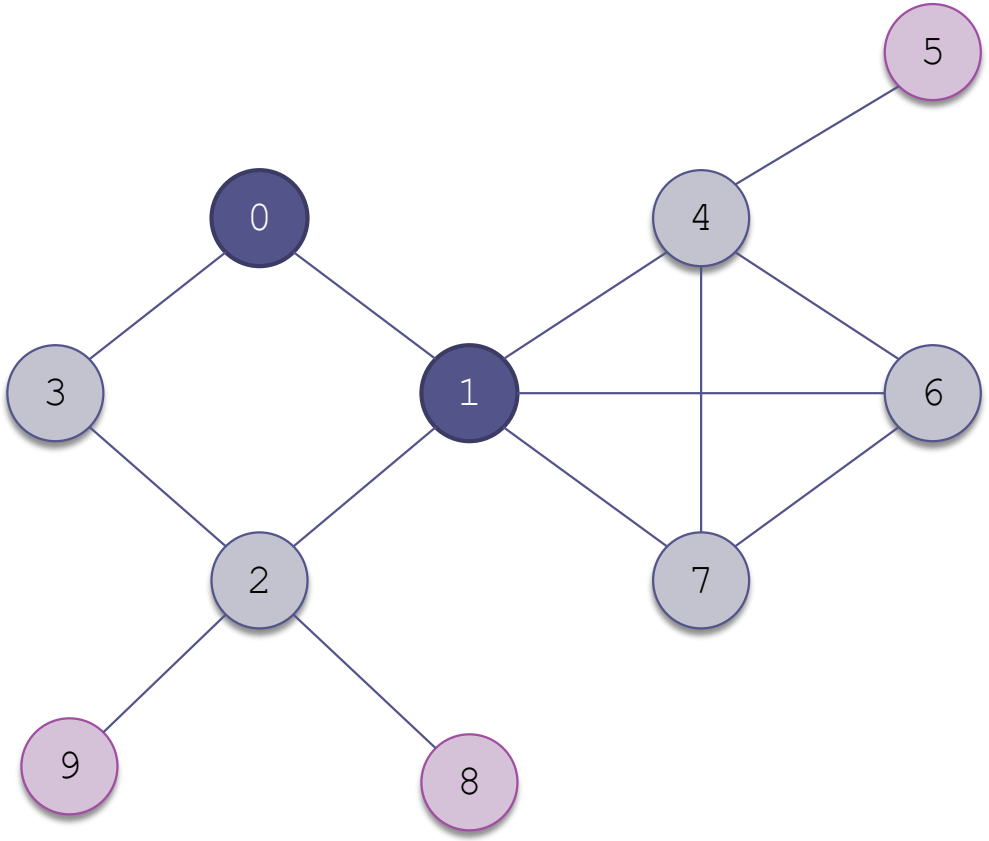
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

and then visit the next node in the queue, 3 (which was identified in the first selection)

Queue:
3, 2, 4, 6, 7

Visit sequence:
0, 1



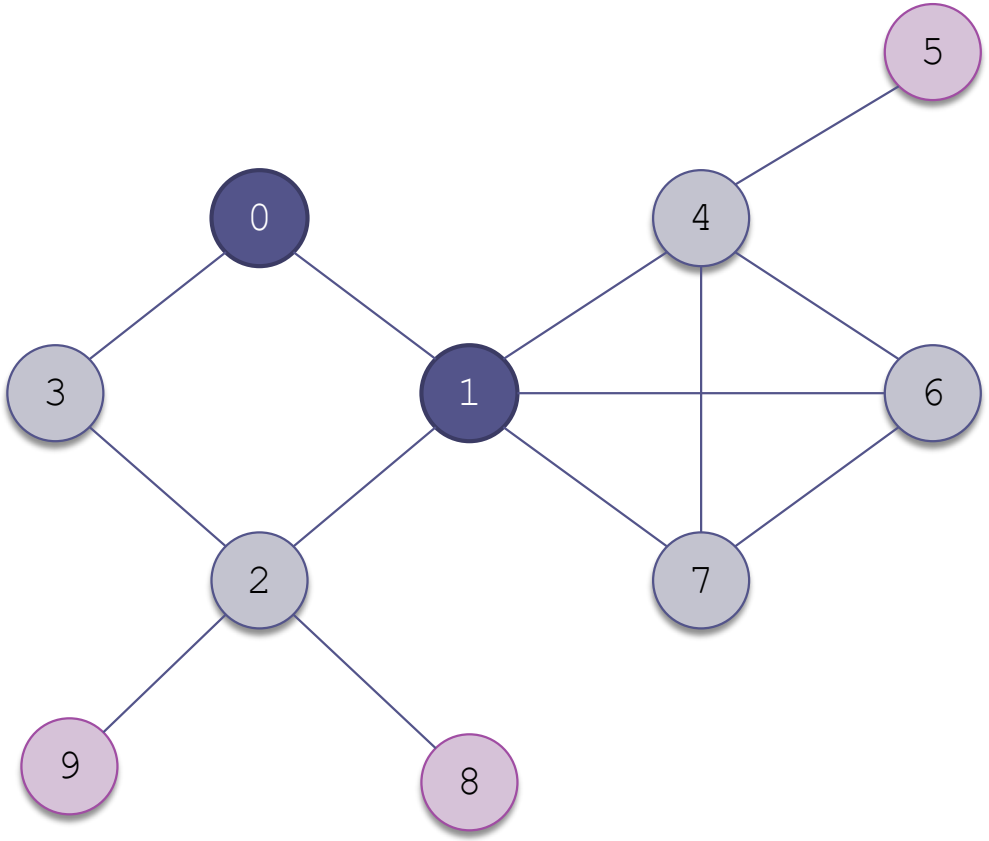
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

and then visit the next node in the queue, 3 (which was identified in the first selection)

Queue:
2, 4, 6, 7

Visit sequence:
0, 1, 3



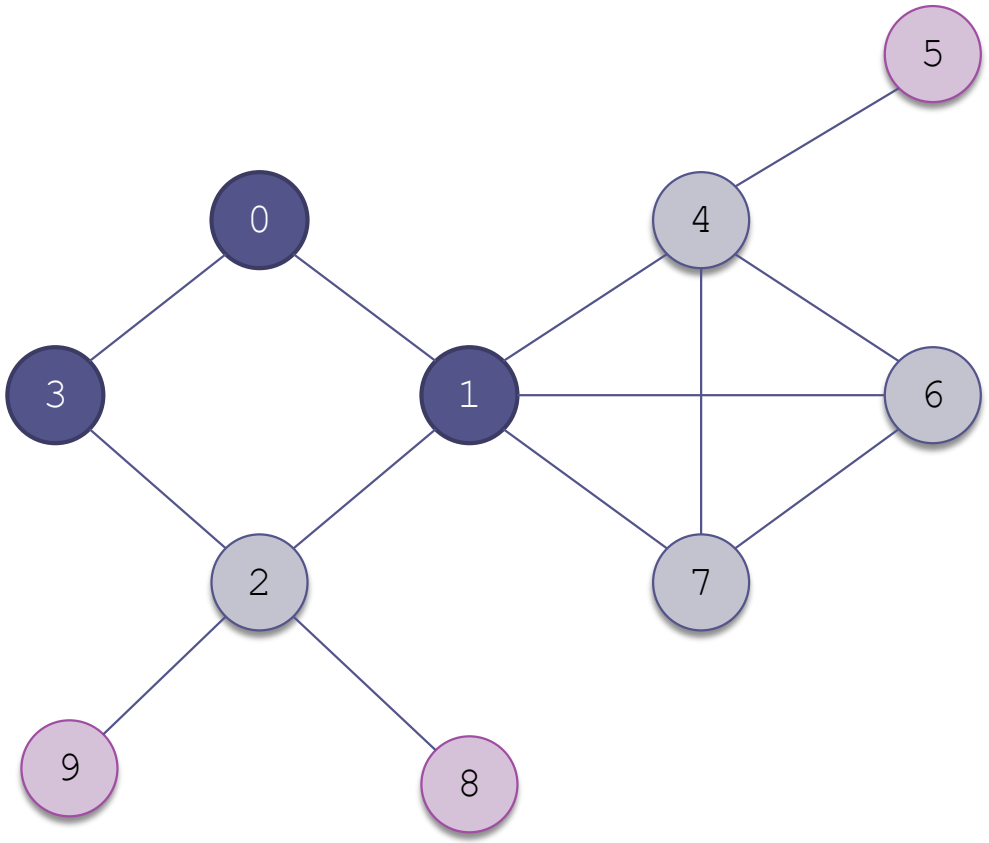
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

3 has two adjacent vertices. 0 has already been visited and 2 has already been identified. We are done with 3

Queue:
2, 4, 6, 7

Visit sequence:
0, 1, 3



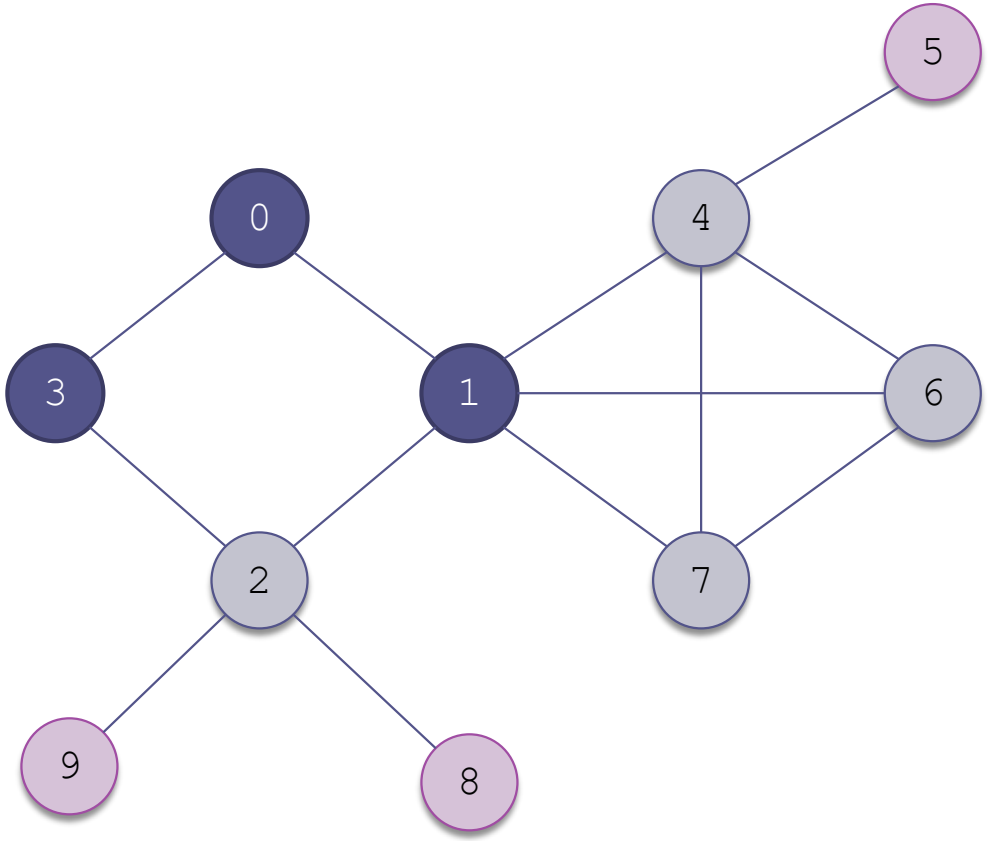
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

The next node in the queue is 2

Queue:
2, 4, 6, 7

Visit sequence:
0, 1, 3



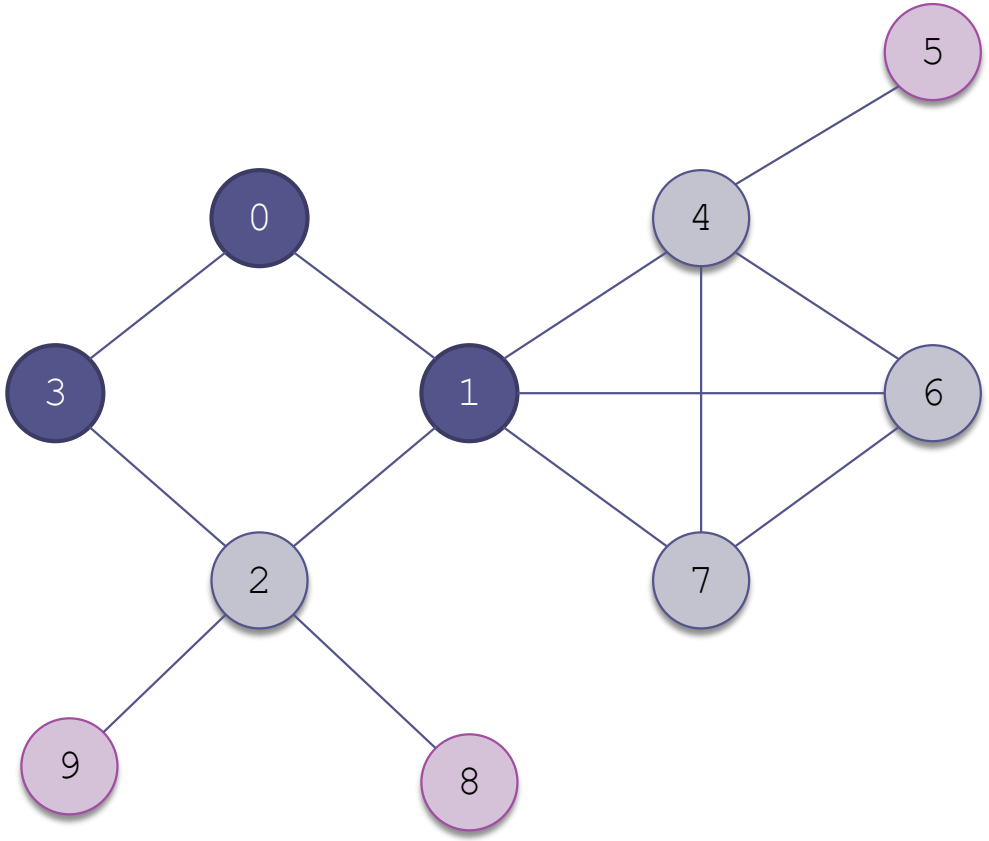
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

The next node in the queue is 2

Queue:
4, 6, 7

Visit sequence:
0, 1, 3, 2



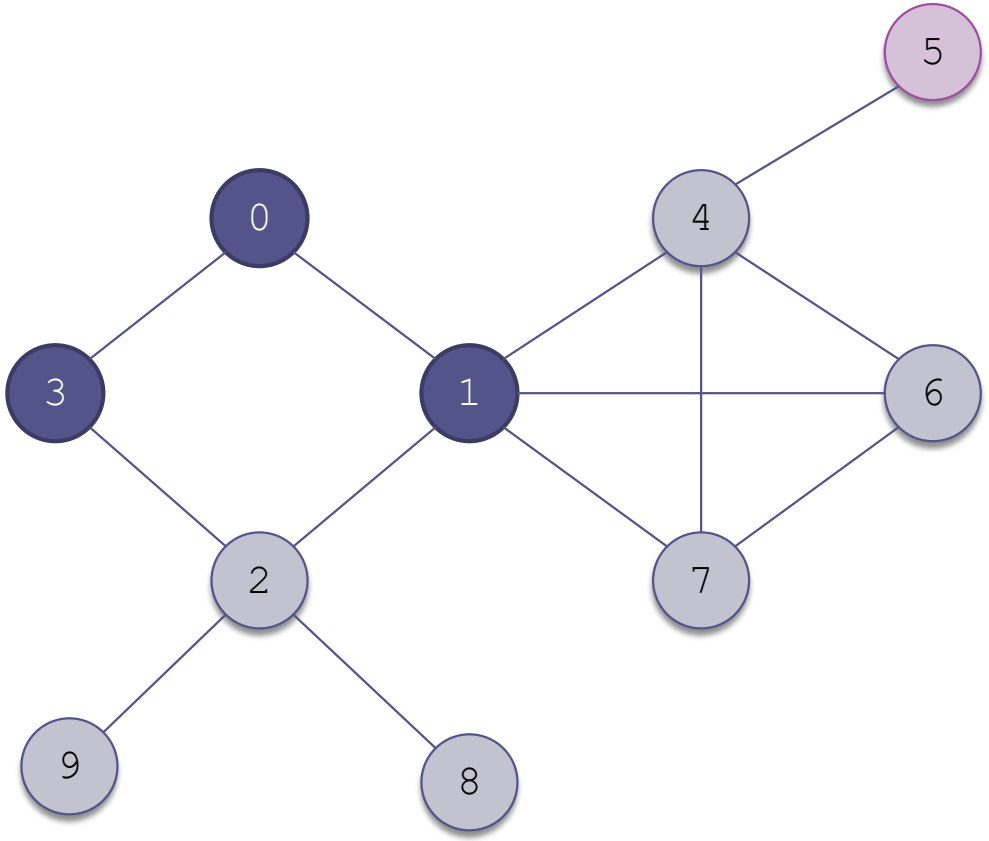
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

8 and 9 are the only adjacent vertices not already visited or identified

Queue:
4, 6, 7, 8, 9

Visit sequence:
0, 1, 3, 2



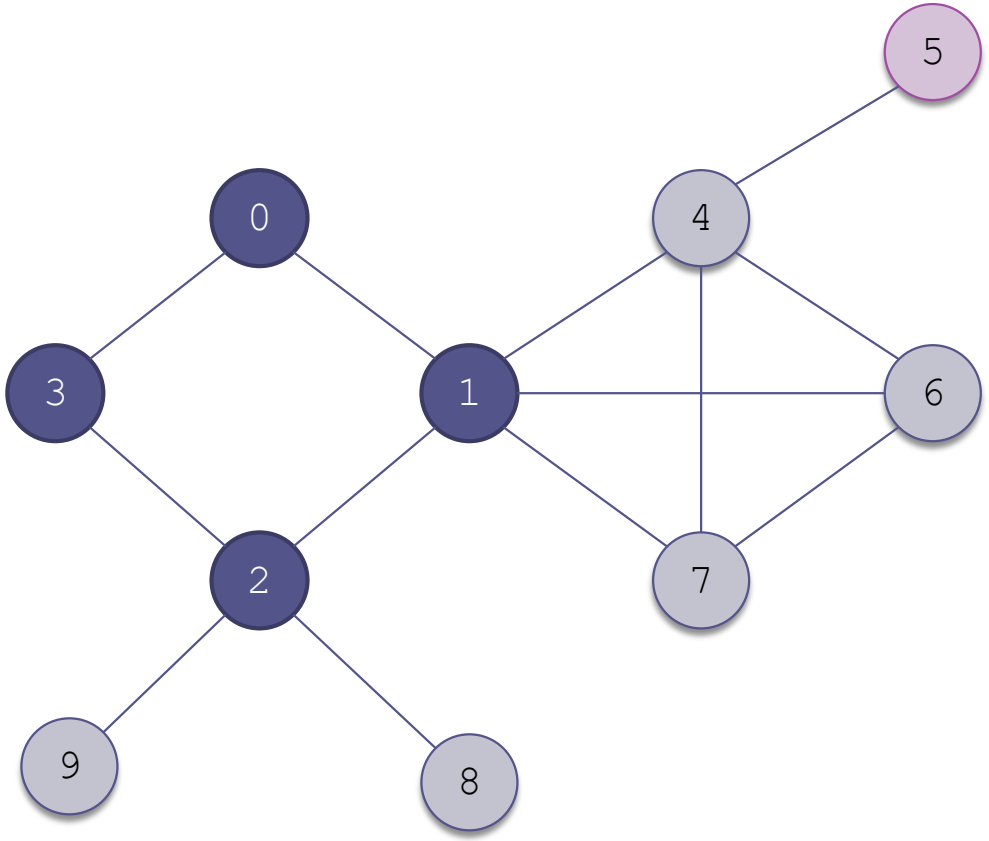
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

4 is next

Queue:
6, 7, 8, 9

Visit sequence:
0, 1, 3, 2, 4



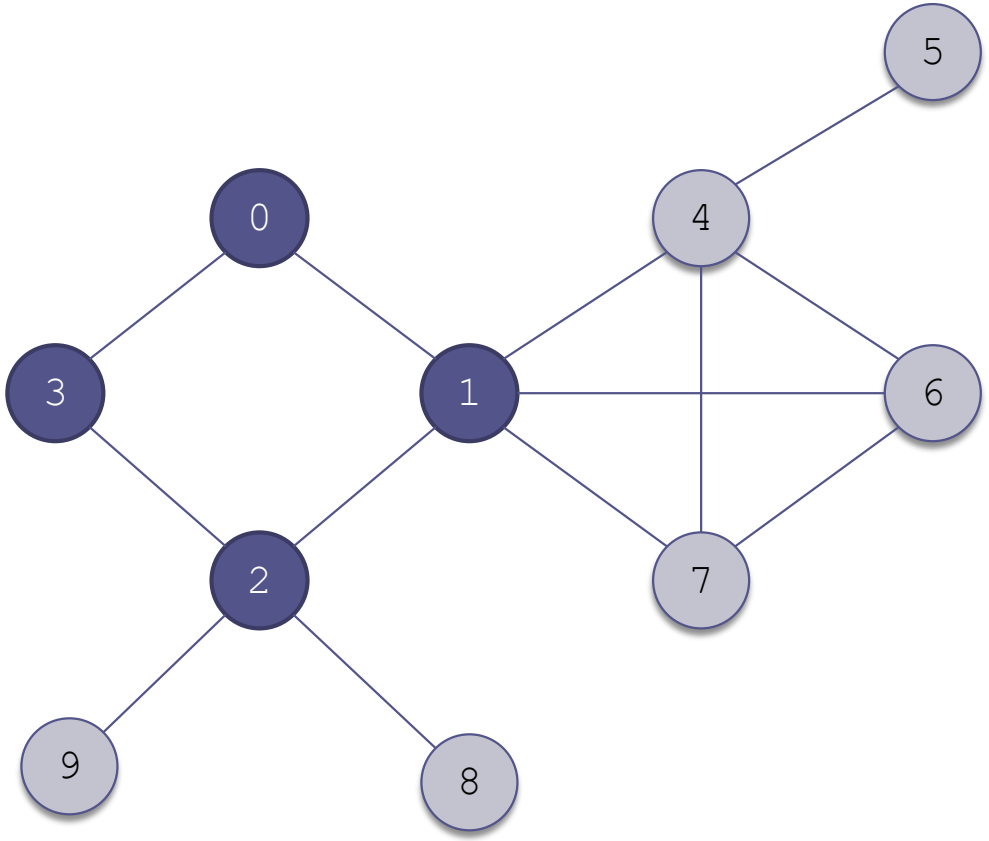
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

5 is the only vertex not already visited or identified

Queue:
6, 7, 8, 9, 5

Visit sequence:
0, 1, 3, 2, 4



0 unvisited

0 visited

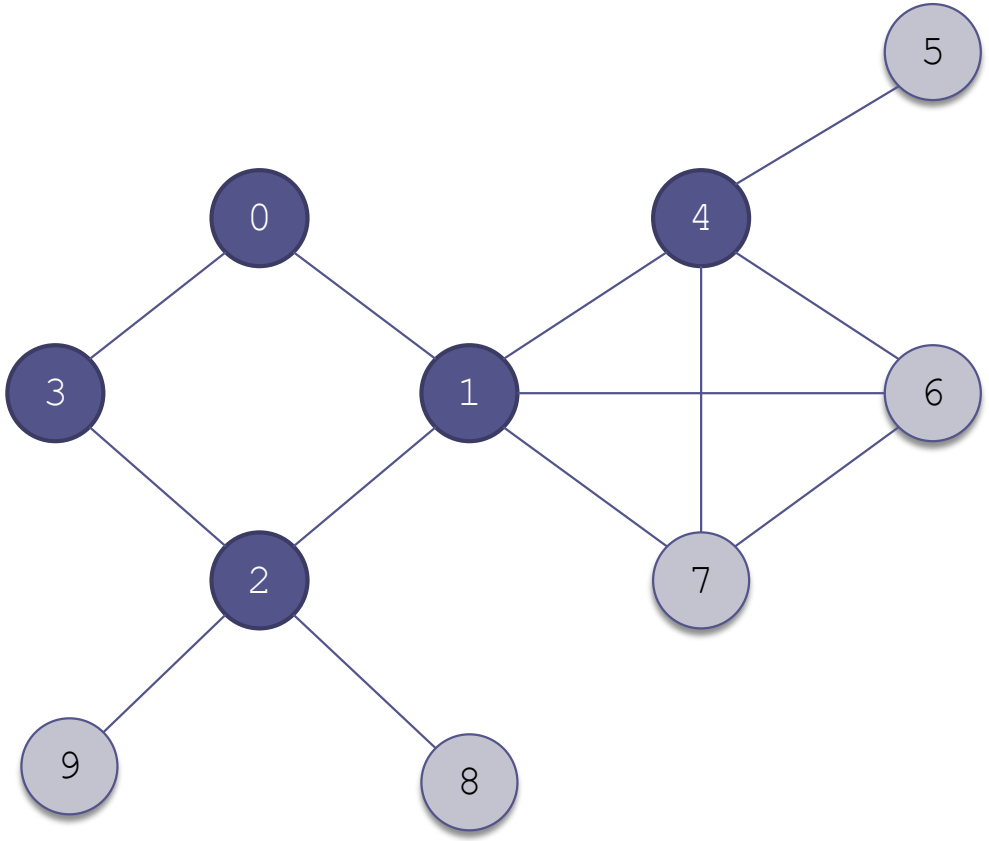
0 identified

Example of a Breadth-First Search (cont.)

6 has no vertices not already visited or identified

Queue:
7, 8, 9, 5

Visit sequence:
0, 1, 3, 2, 4, 6



0 unvisited

0 visited

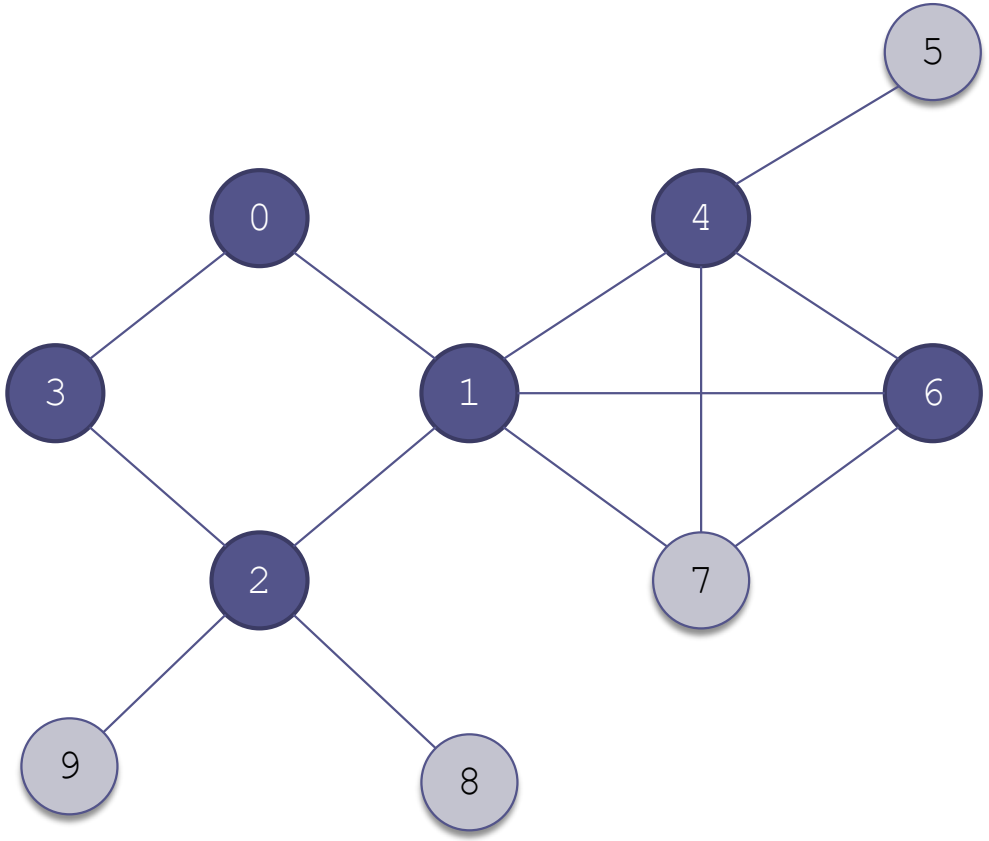
0 identified

Example of a Breadth-First Search (cont.)

6 has no vertices not already visited or identified

Queue:
7, 8, 9, 5

Visit sequence:
0, 1, 3, 2, 4, 6



0 unvisited

0 visited

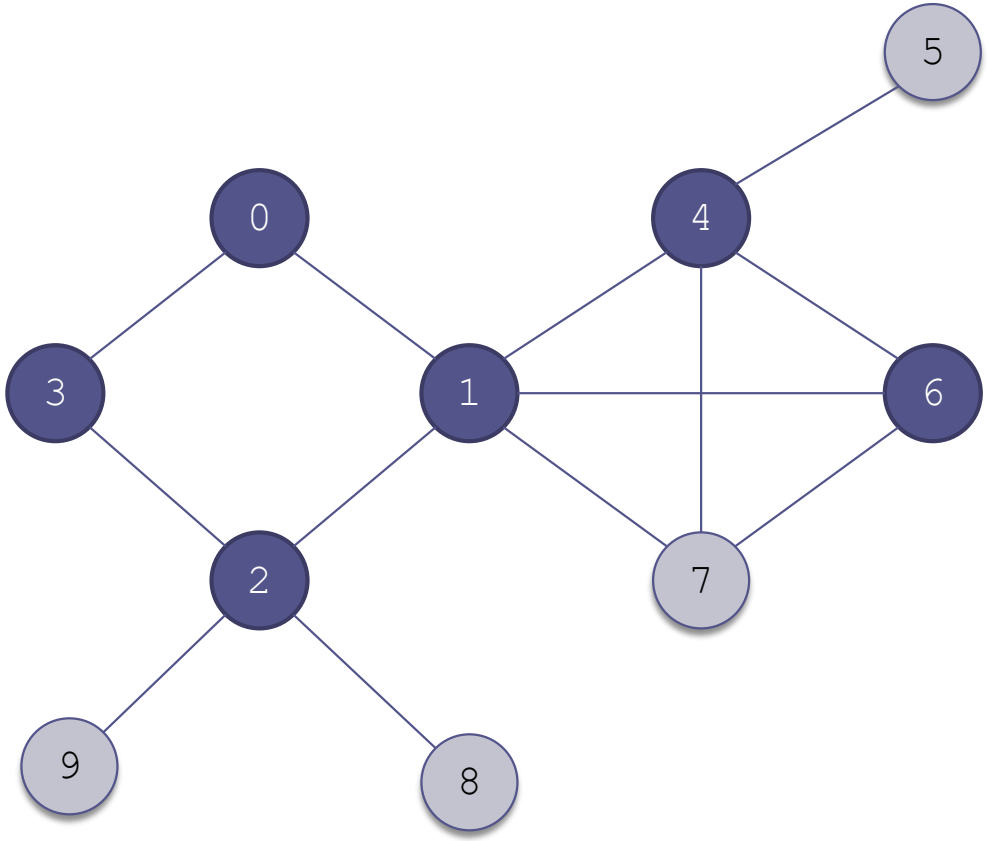
0 identified

Example of a Breadth-First Search (cont.)

7 has no vertices not already visited or identified

Queue:
8, 9, 5

Visit sequence:
0, 1, 3, 2, 4, 6, 7



0 unvisited

0 visited

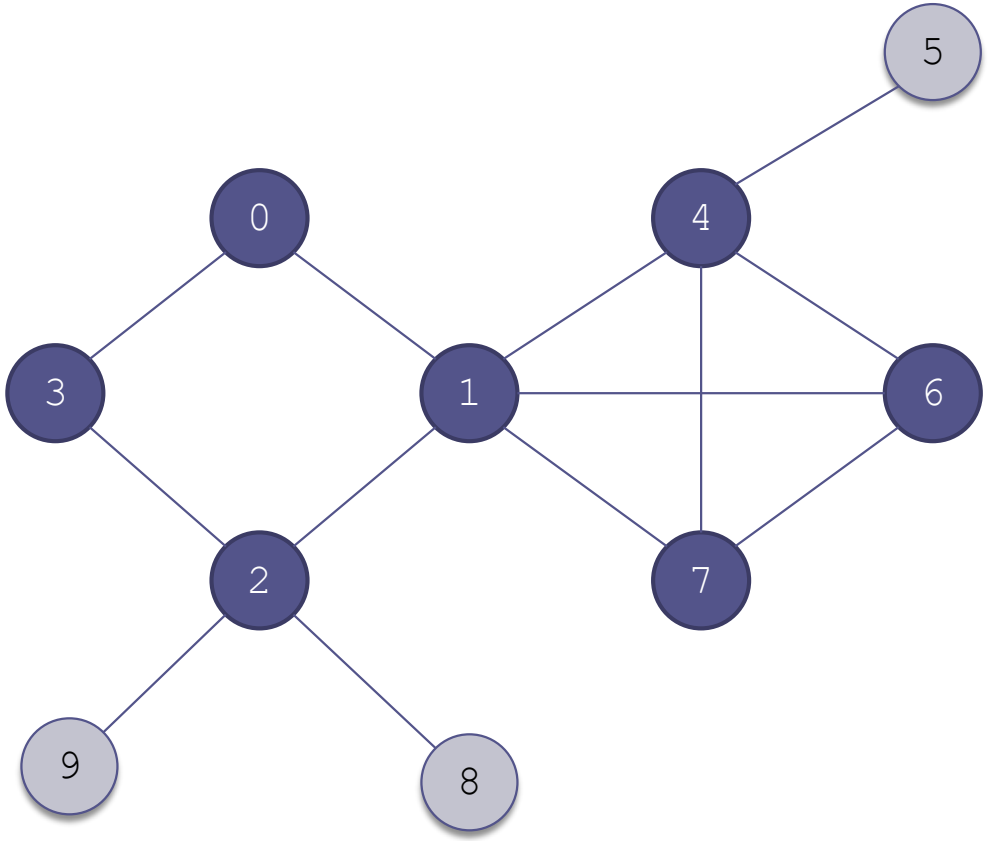
0 identified

Example of a Breadth-First Search (cont.)

7 has no vertices not already visited or identified

Queue:
8, 9, 5

Visit sequence:
0, 1, 3, 2, 4, 6, 7



0 unvisited

0 visited

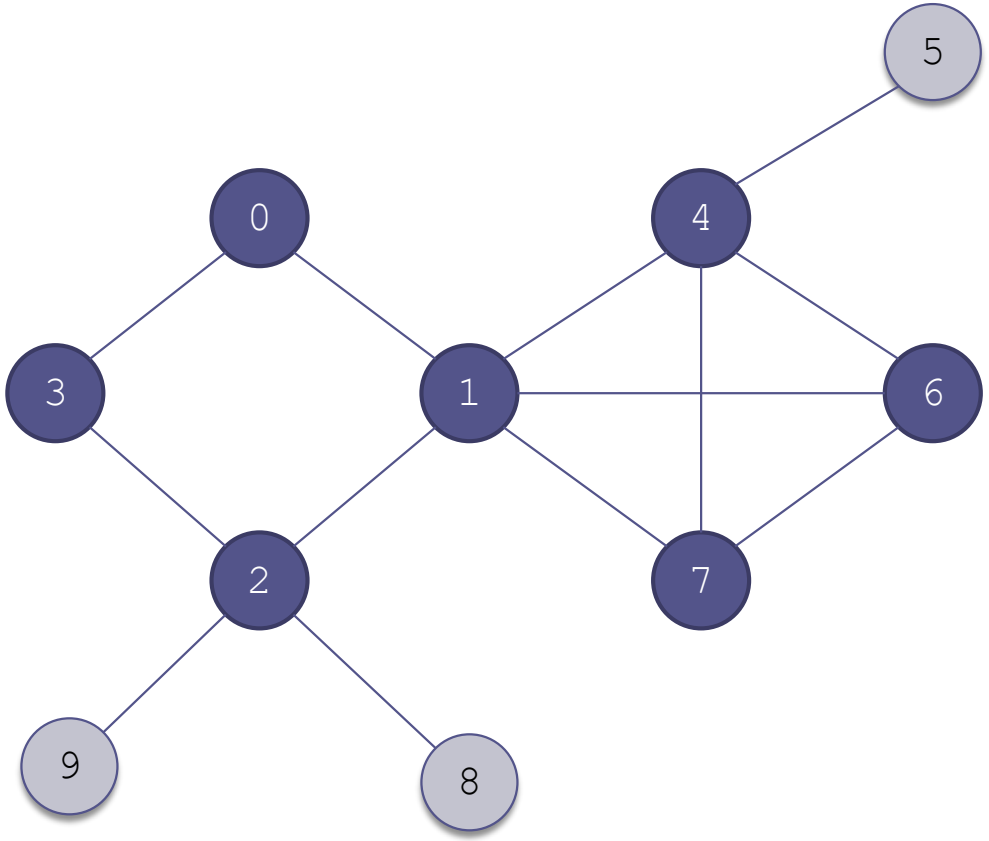
0 identified

Example of a Breadth-First Search (cont.)

We go back to the vertices of 2 and visit them

Queue:
8, 9, 5

Visit sequence:
0, 1, 3, 2, 4, 6, 7



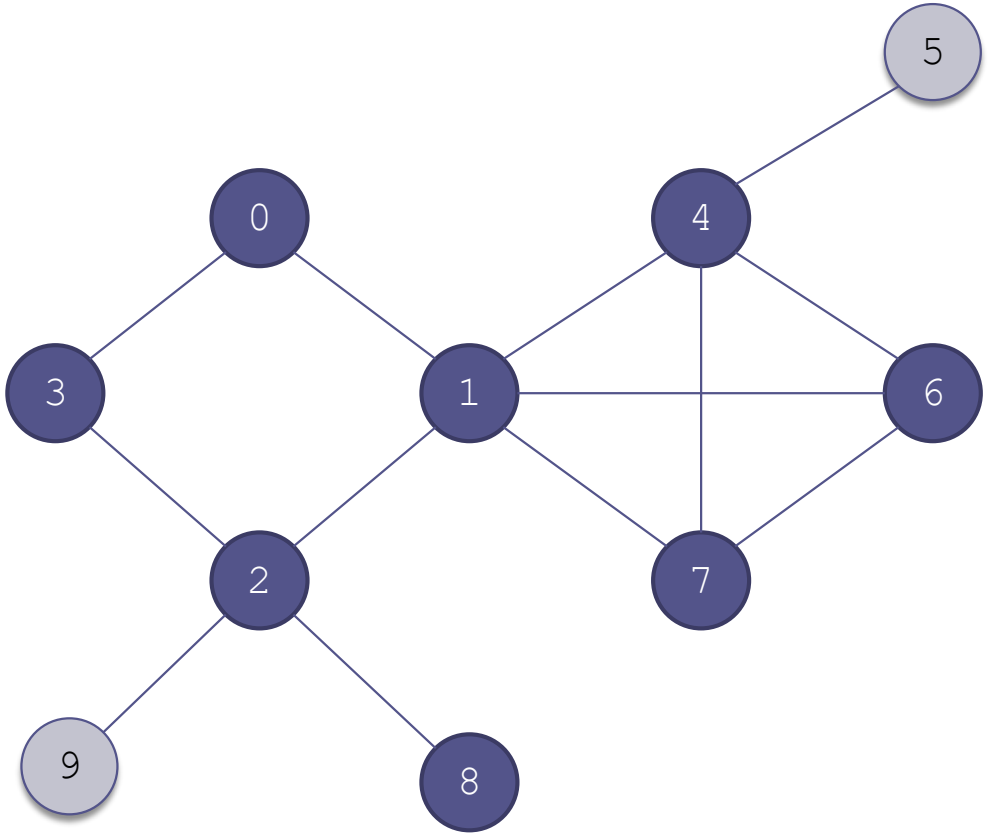
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

8 has no vertices not already visited or identified

Queue:
9, 5

Visit sequence:
0, 1, 3, 2, 4, 6, 7, 8



0 unvisited

0 visited

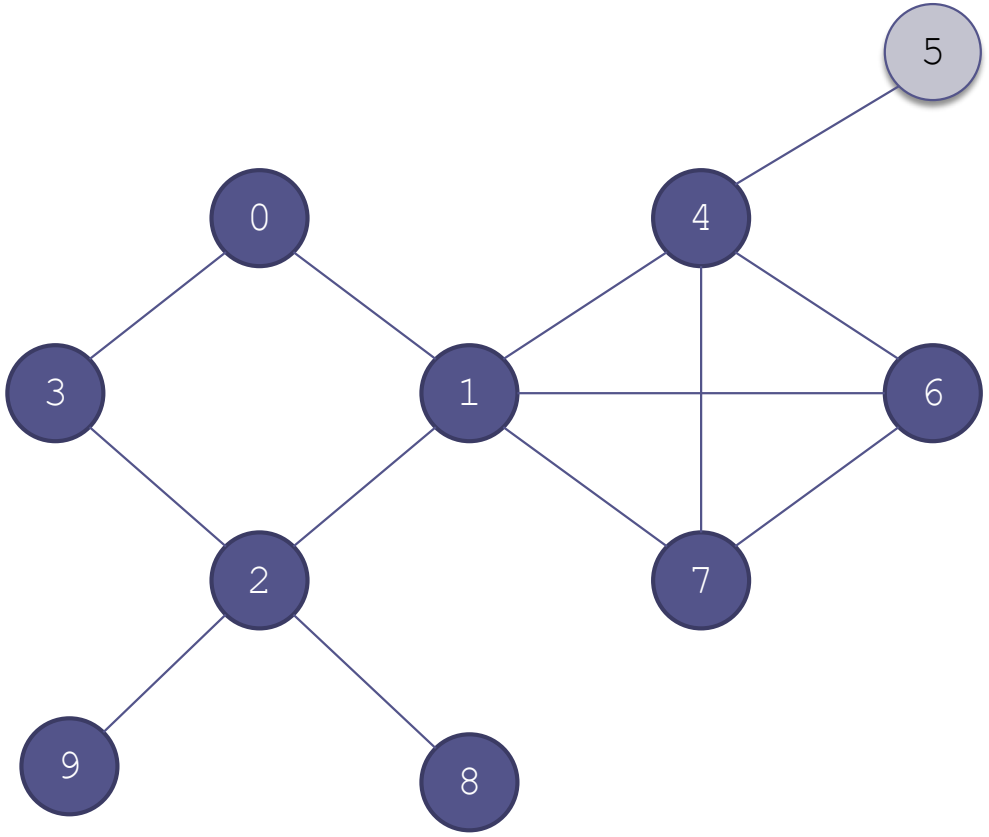
0 identified

Example of a Breadth-First Search (cont.)

9 has no vertices not already visited or identified

Queue:
5

Visit sequence:
0, 1, 3, 2, 4, 6, 7, 8, 9



0 unvisited

0 visited

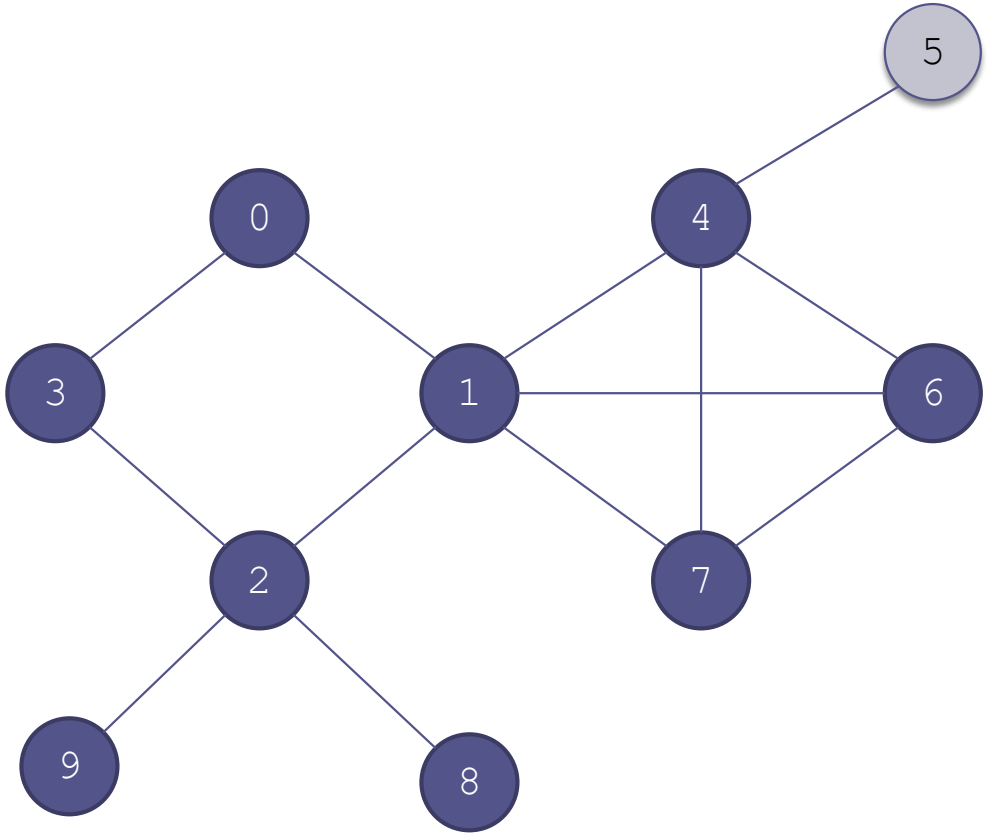
0 identified

Example of a Breadth-First Search (cont.)

Finally we visit *5*

Queue:
5

Visit sequence:
0, 1, 3, 2, 4, 6, 7, 8, 9



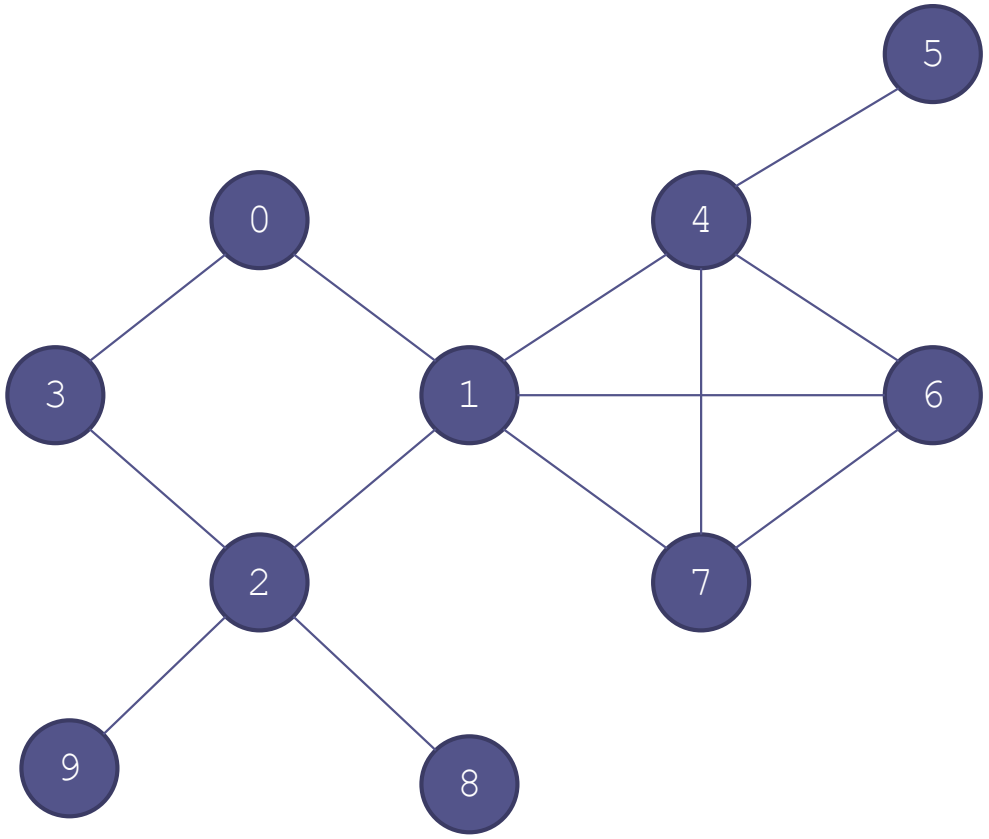
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

which has no vertices not already visited or identified

Queue:
empty

Visit sequence:
0, 1, 3, 2, 4, 6, 7, 8, 9, 5



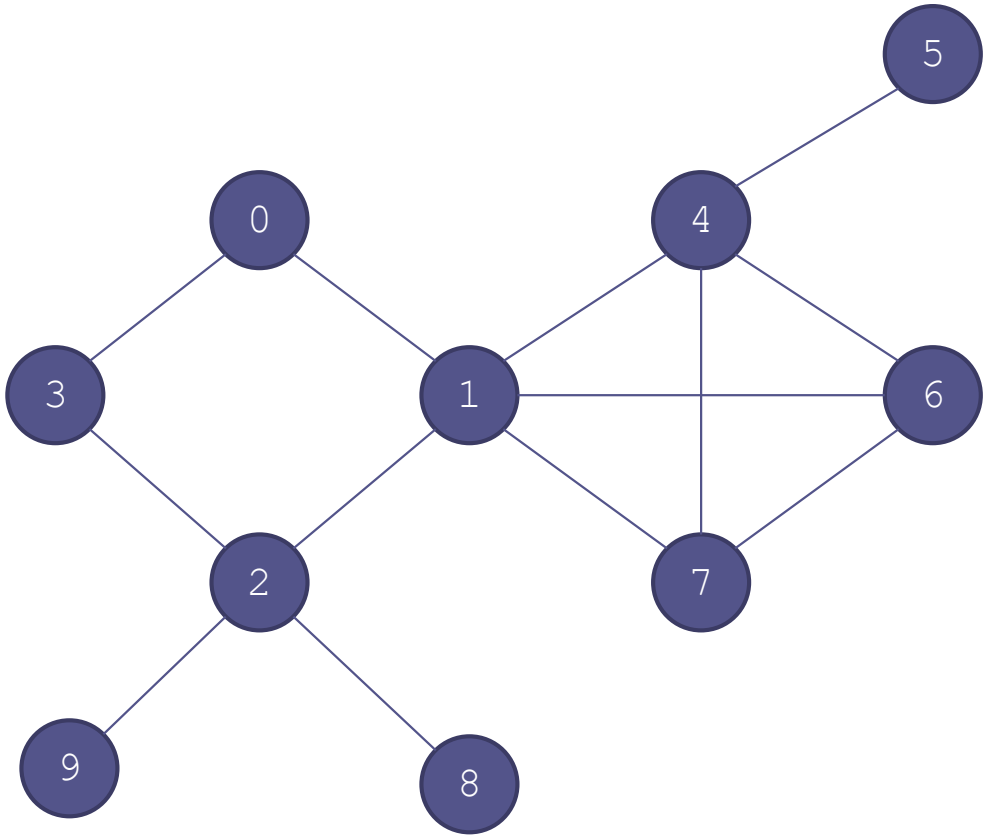
0 unvisited 0 visited 0 identified

Example of a Breadth-First Search (cont.)

The queue is empty; all vertices have been visited

Queue:
empty

Visit sequence:
0, 1, 3, 2, 4, 6, 7, 8, 9, 5



0 unvisited 0 visited 0 identified

Algorithm for Breadth-First Search

Algorithm for Breadth-First Search

1. Take an arbitrary start vertex, mark it identified (color it light blue), and place it in a queue.
2. `while` the queue is not empty
3. Take a vertex, u, out of the queue and visit u.
4. `for` all vertices, v, adjacent to this vertex, u
5. `if` v has not been identified or visited
6. Mark it identified (color it light blue).
7. Insert vertex v into the queue.
8. We are now finished visiting u (color it dark blue).

Algorithm for Breadth-First Search (cont.)

- We can save the information we need to represent the tree by storing the parent of each vertex when we identify it
- We can refine Step 7 of the algorithm to accomplish this:

    7.1 Insert vertex *v* into the queue

    7.2 Set the parent of *v* to *u*

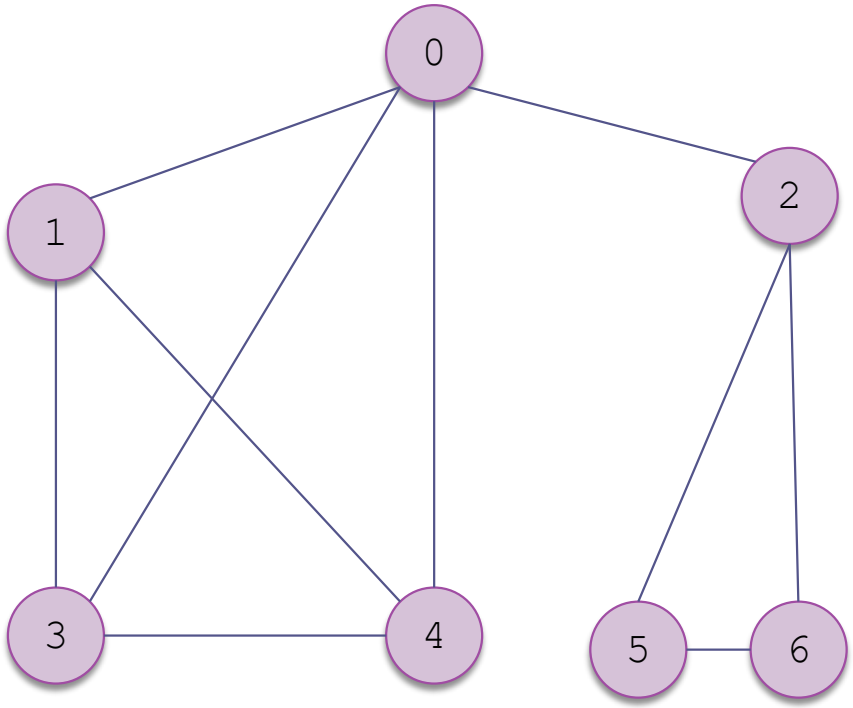
Performance Analysis of Breadth-First Search

- The loop at Step 2 is performed for each vertex.
- The inner loop at Step 4 is performed for $|E_v|$, the number of edges that originate at that vertex)
- The total number of steps is the sum of the edges that originate at each vertex, which is the total number of edges
- The algorithm is $O(|E|)$

Depth-First Search

- In a depth-first search,
  - start at a vertex,
  - visit it,
  - choose one adjacent vertex to visit;
  - then, choose a vertex adjacent to that vertex to visit,
  - and so on until you go no further;
  - then back up and see whether a new vertex can be found

Example of a Depth-First Search



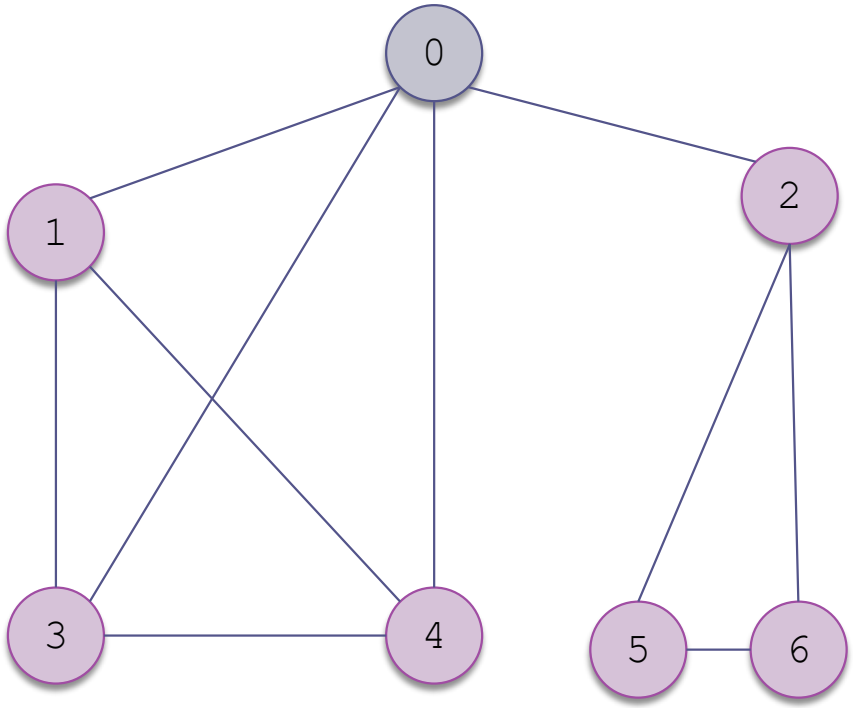
0 unvisited

0 visited

0 being visited

Example of a Depth-First Search (cont.)

Mark 0 as being visited

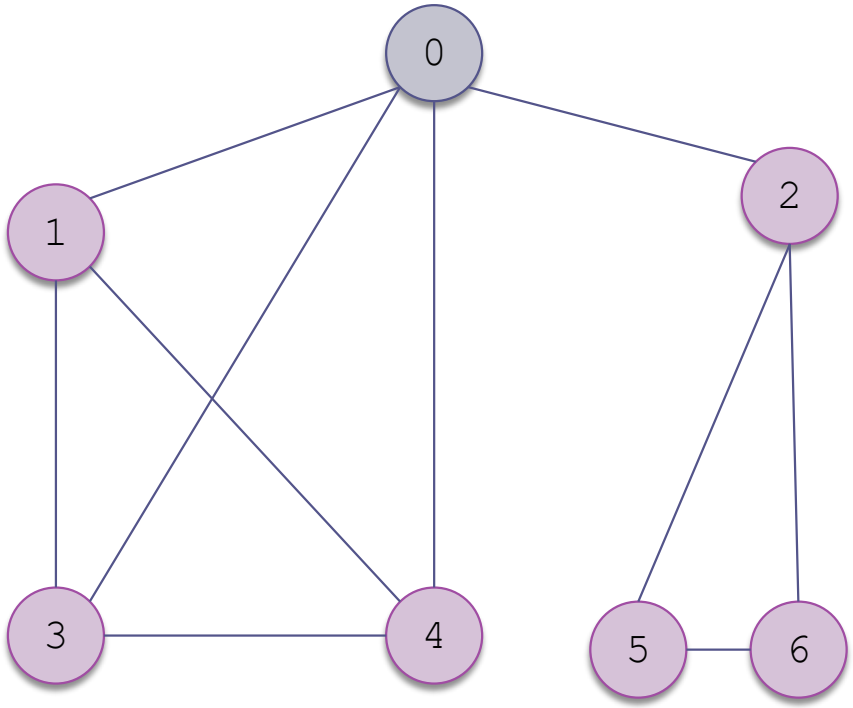


Discovery (Visit) order:
0

Finish order:

Example of a Depth-First Search (cont.)

Choose an adjacent vertex that is not being visited

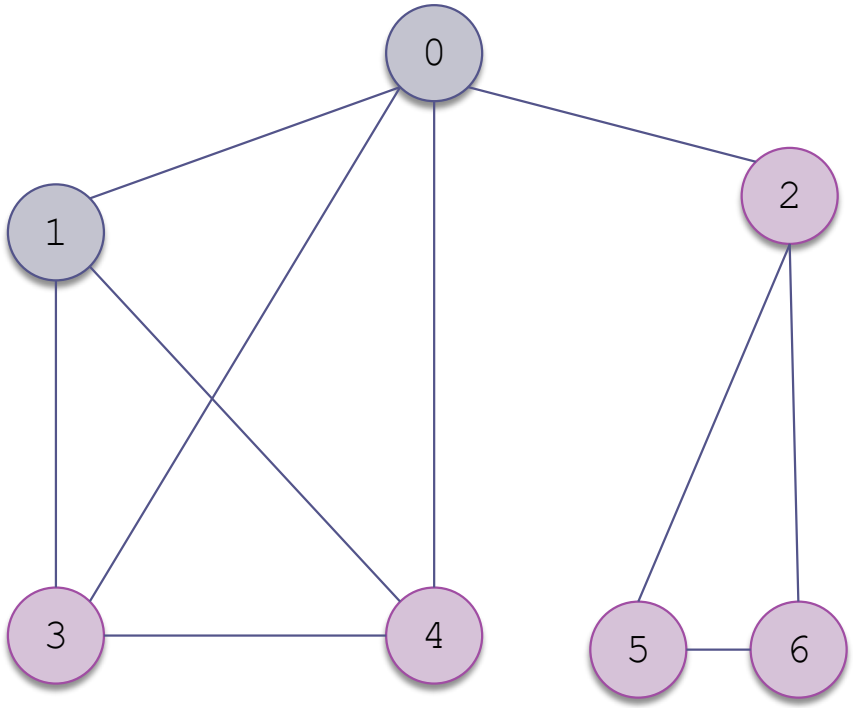


Discovery (Visit) order:
0

Finish order:

Example of a Depth-First Search (cont.)

Choose an adjacent vertex that is not being visited

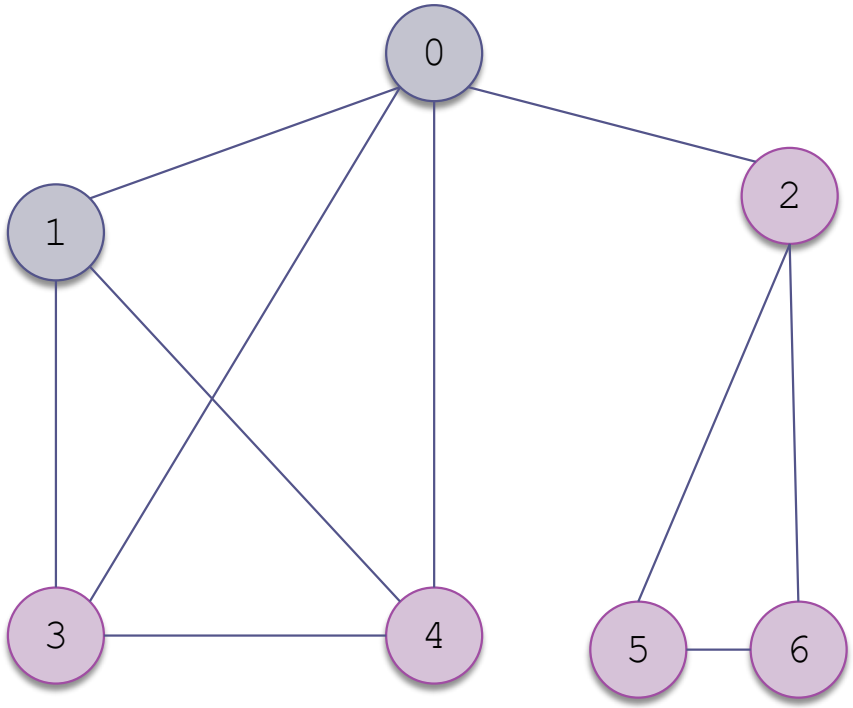


Discovery (Visit) order:
0, 1

Finish order:

Example of a Depth-First Search (cont.)

(Recursively) choose an adjacent vertex that is not being visited

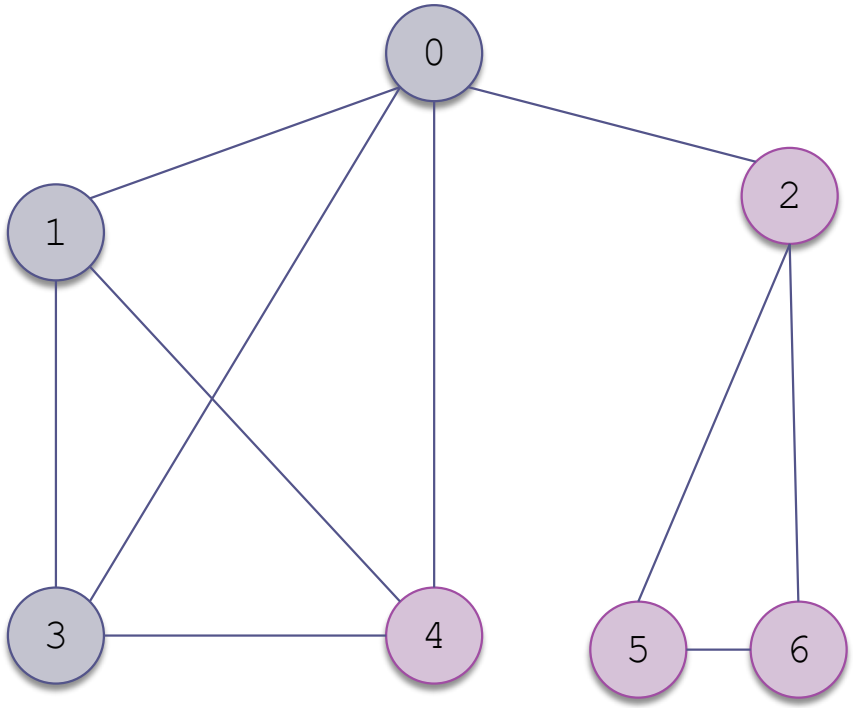


Discovery (Visit) order:
0, 1, 3

Finish order:

Example of a Depth-First Search (cont.)

(Recursively) choose an adjacent vertex that is not being visited

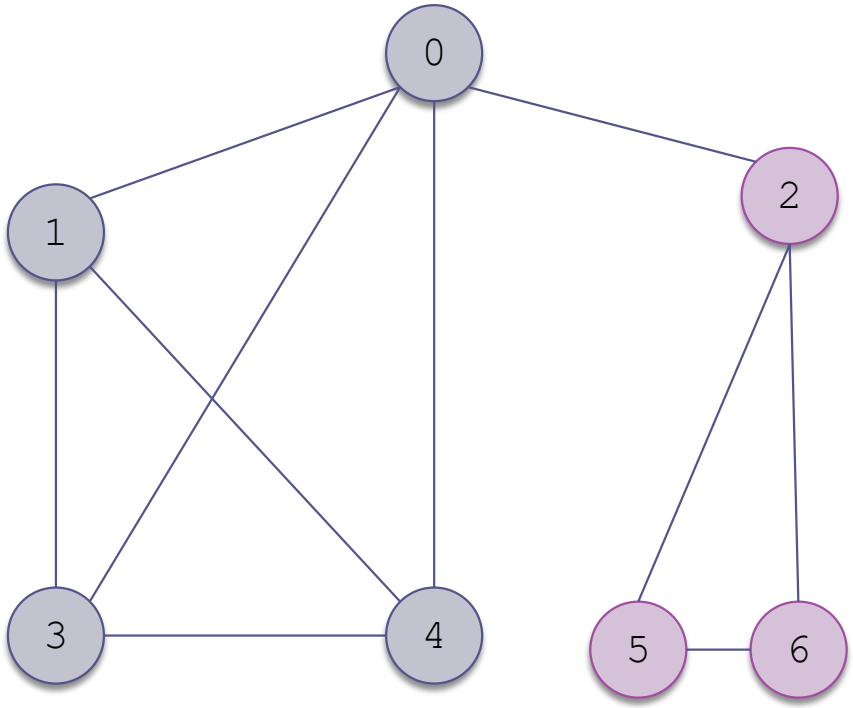


Discovery (Visit) order:
0, 1, 3

Finish order:

Example of a Depth-First Search (cont.)

(Recursively) choose an adjacent vertex that is not being visited

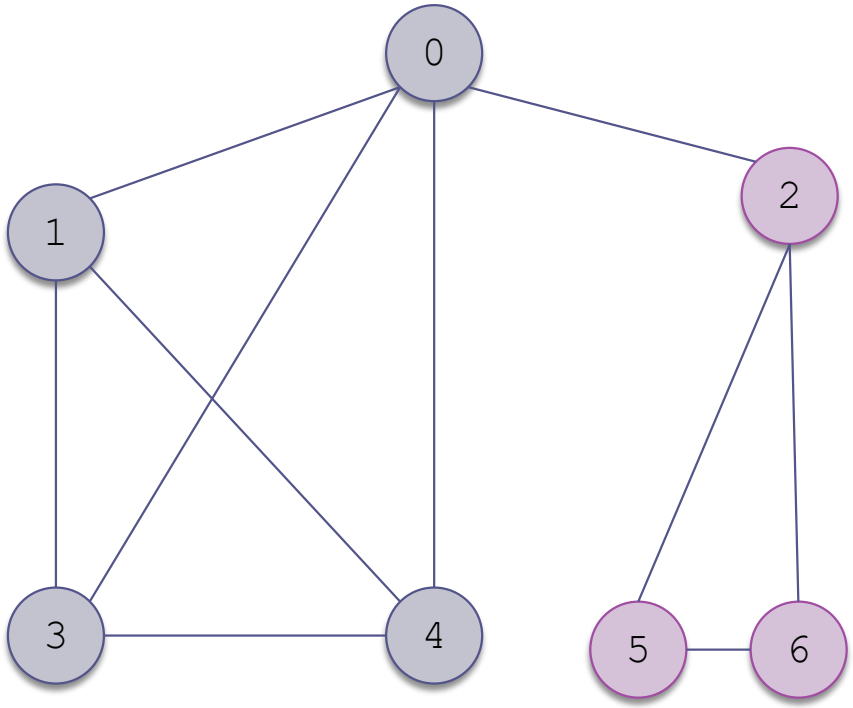


Discovery (Visit) order:
0, 1, 3, 4

Finish order:

Example of a Depth-First Search (cont.)

There are no vertices adjacent to 4 that are not being visited

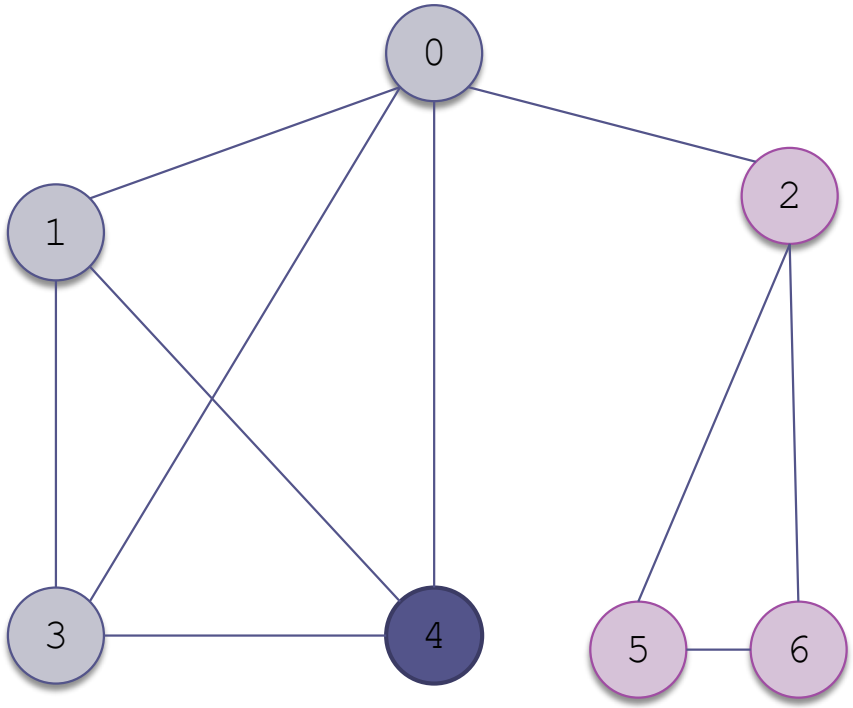


Discovery (Visit) order:
0, 1, 3, 4

Finish order:

Example of a Depth-First Search (cont.)

Mark 4 as visited

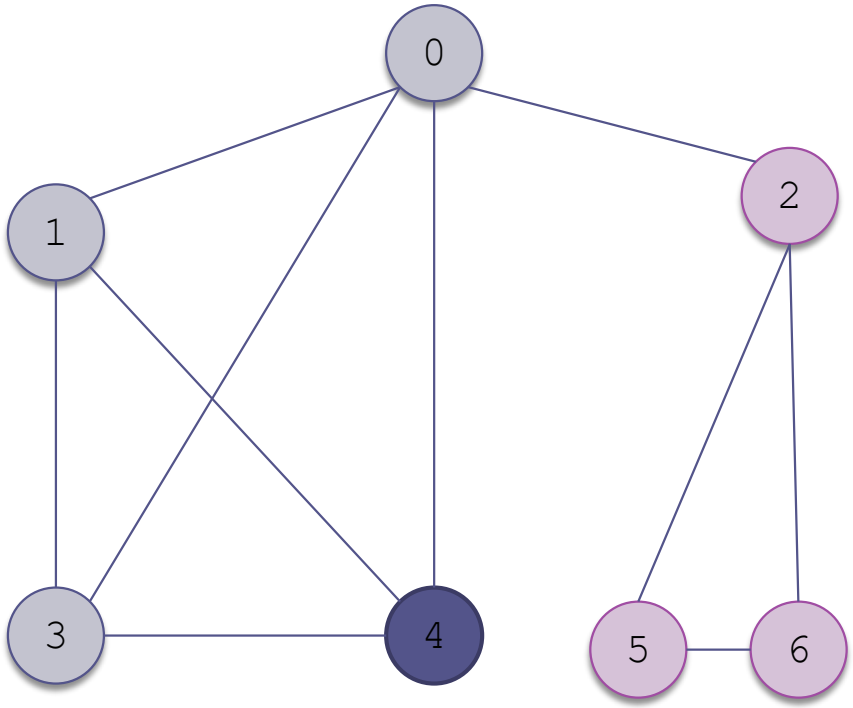


Discovery (Visit) order:
0, 1, 3, 4

Finish order:
4

Example of a Depth-First Search (cont.)

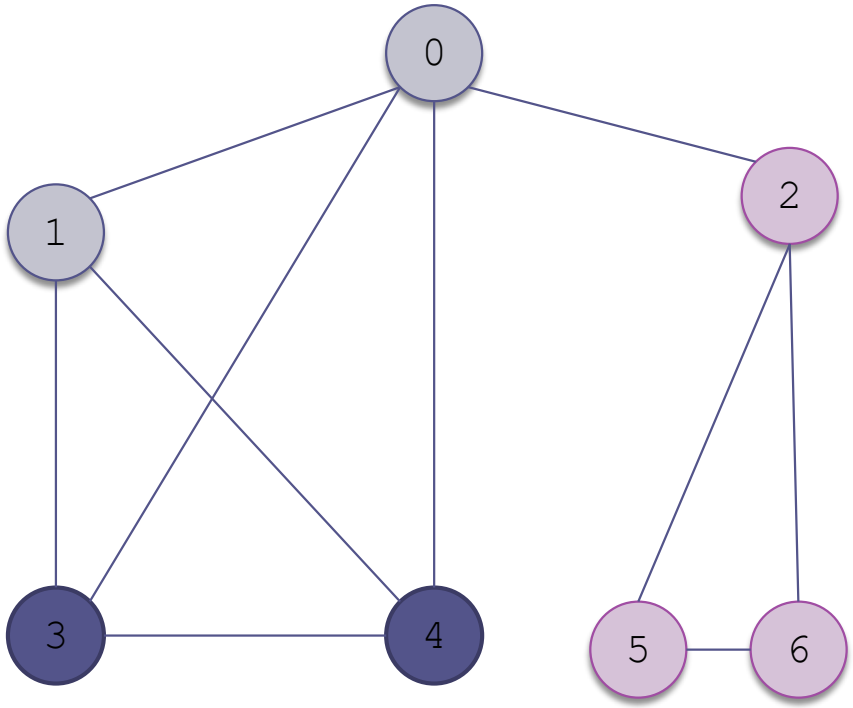
Return from the recursion to 3; all adjacent nodes to 3 are being visited



Finish order:
4

Example of a Depth-First Search (cont.)

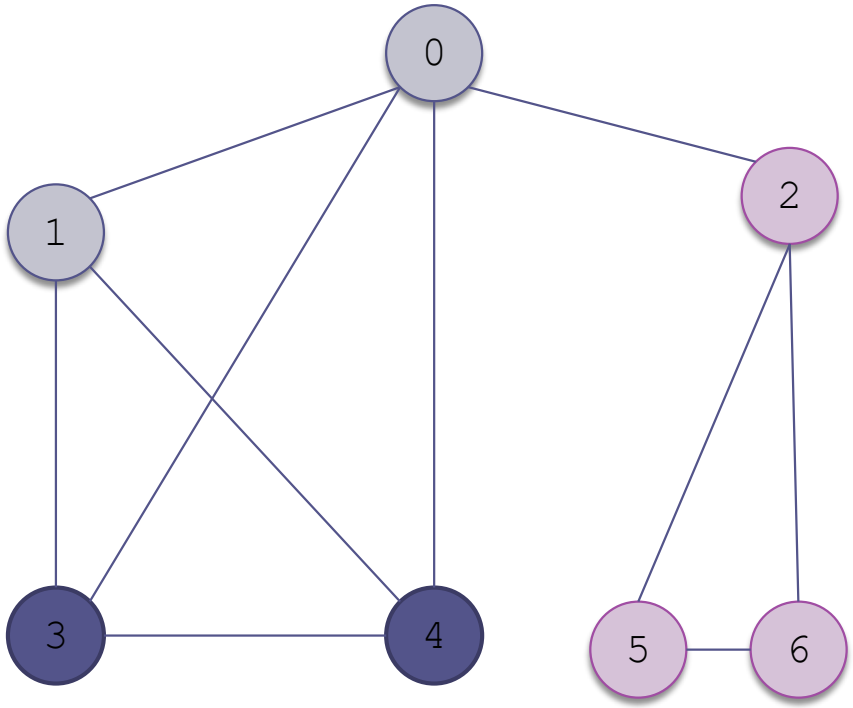
Mark 3 as visited



Finish order:
4, 3

Example of a Depth-First Search (cont.)

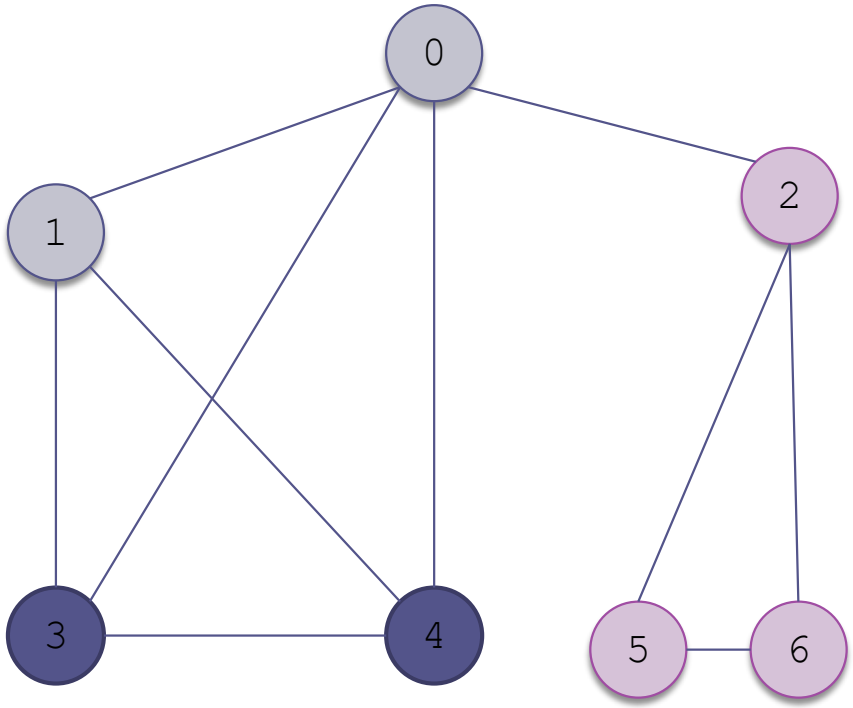
Return from the recursion to 1



Finish order:
4, 3

Example of a Depth-First Search (cont.)

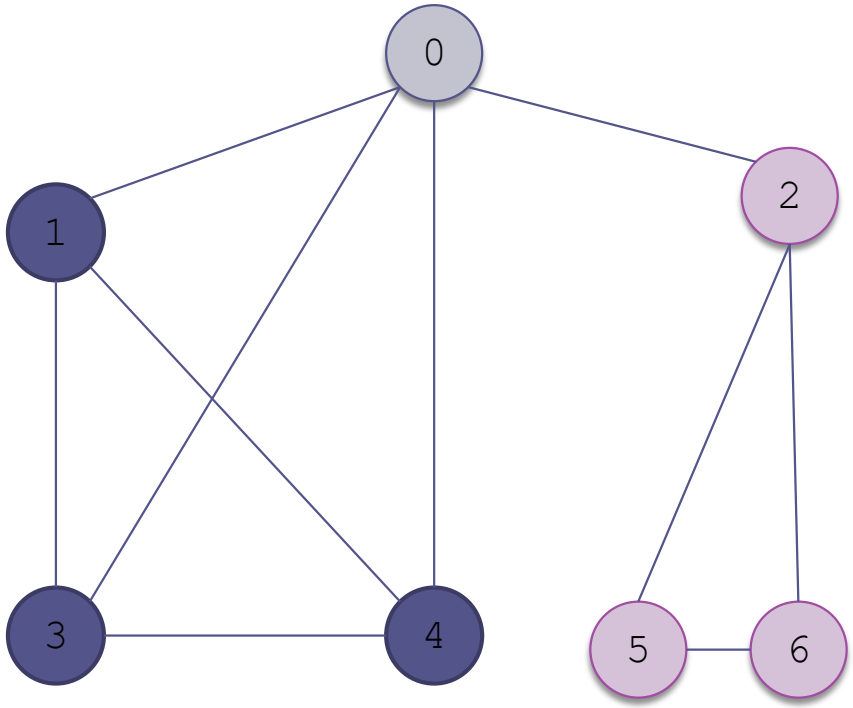
All vertices adjacent to 1 are being visited



Finish order:
4, 3

Example of a Depth-First Search (cont.)

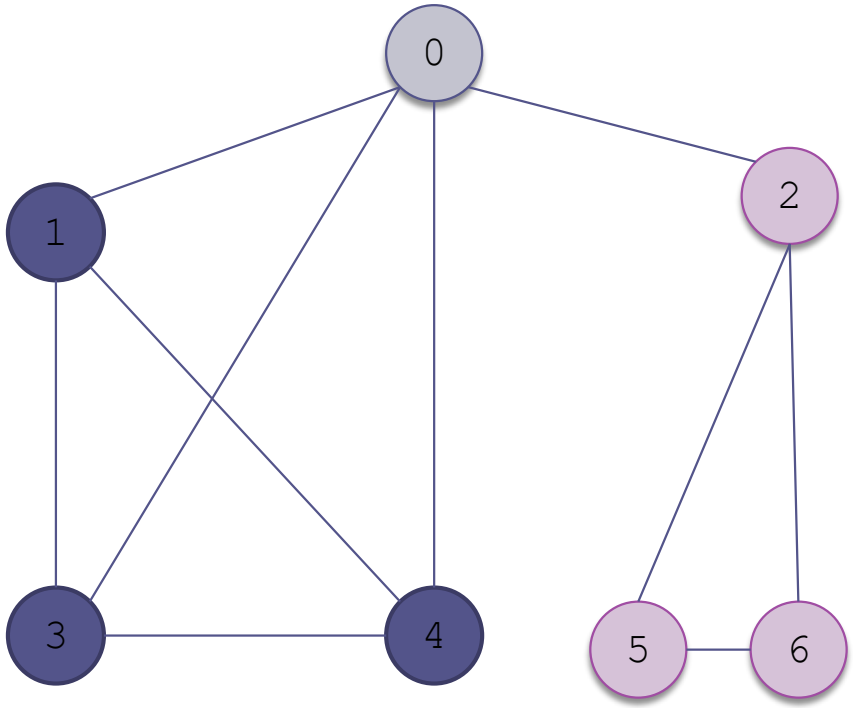
Mark 1 as visited



Finish order:
4, 3, 1

Example of a Depth-First Search (cont.)

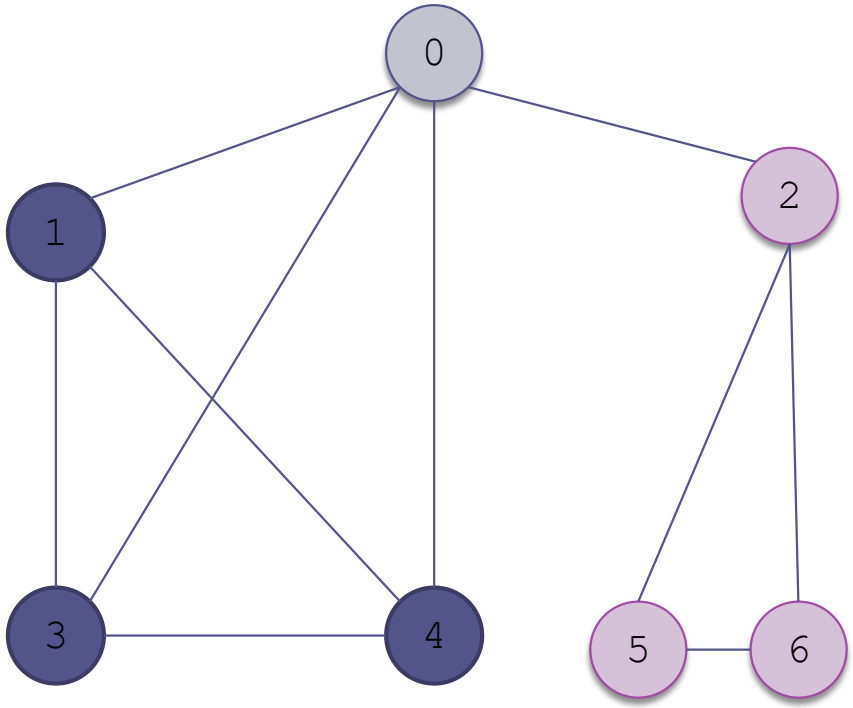
Return from the recursion to 0



Finish order:
4, 3, 1

Example of a Depth-First Search (cont.)

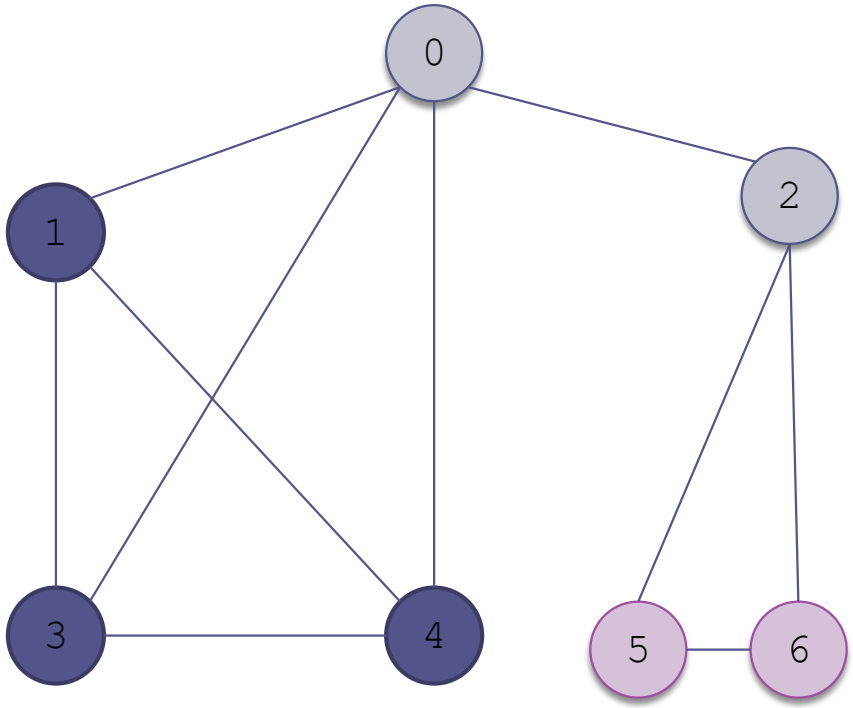
2 is adjacent to 1 and is not being visited



Finish order:
4, 3, 1

Example of a Depth-First Search (cont.)

2 is adjacent to 1 and is not being visited

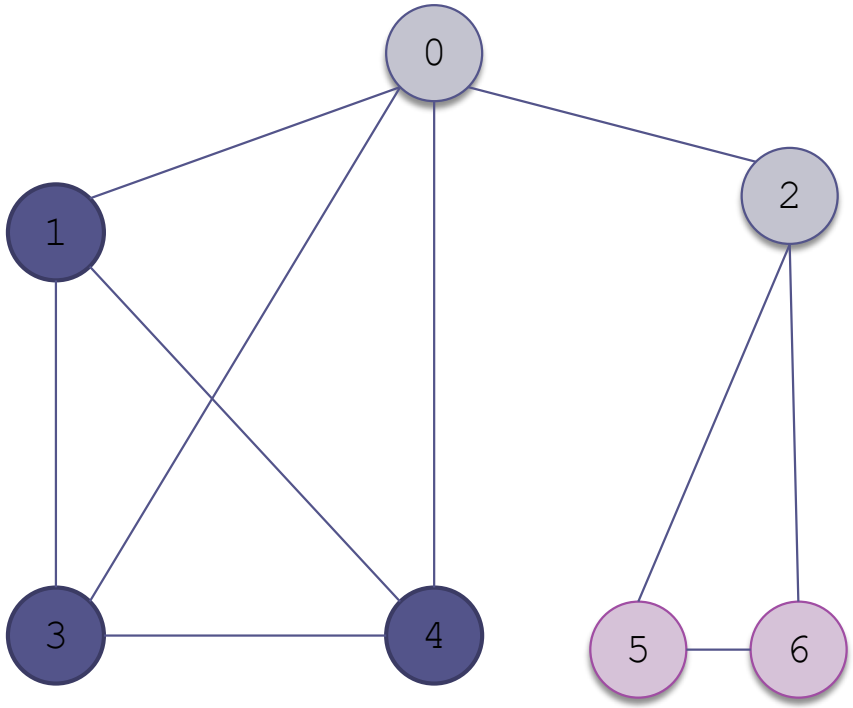


Discovery (Visit) order:
0, 1, 3, 4, 2

Finish order:
4, 3, 1

Example of a Depth-First Search (cont.)

5 is adjacent to 2 and is not being visited

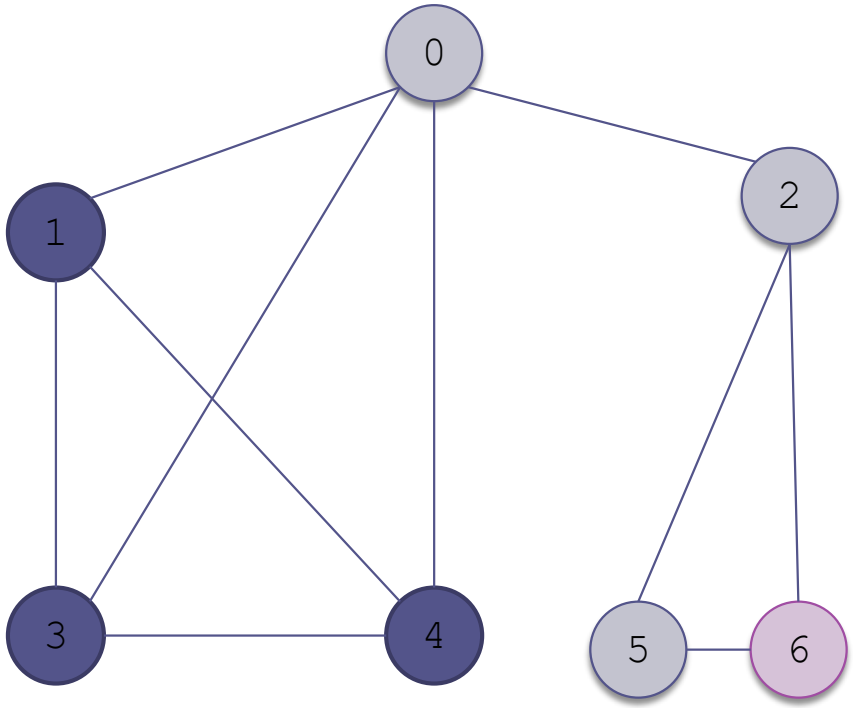


Discovery (Visit) order:
0, 1, 3, 4, 2

Finish order:
4, 3, 1

Example of a Depth-First Search (cont.)

5 is adjacent to 2 and is not being visited



Discovery (Visit) order:
0, 1, 3, 4, 2, 5

Finish order:
4, 3, 1

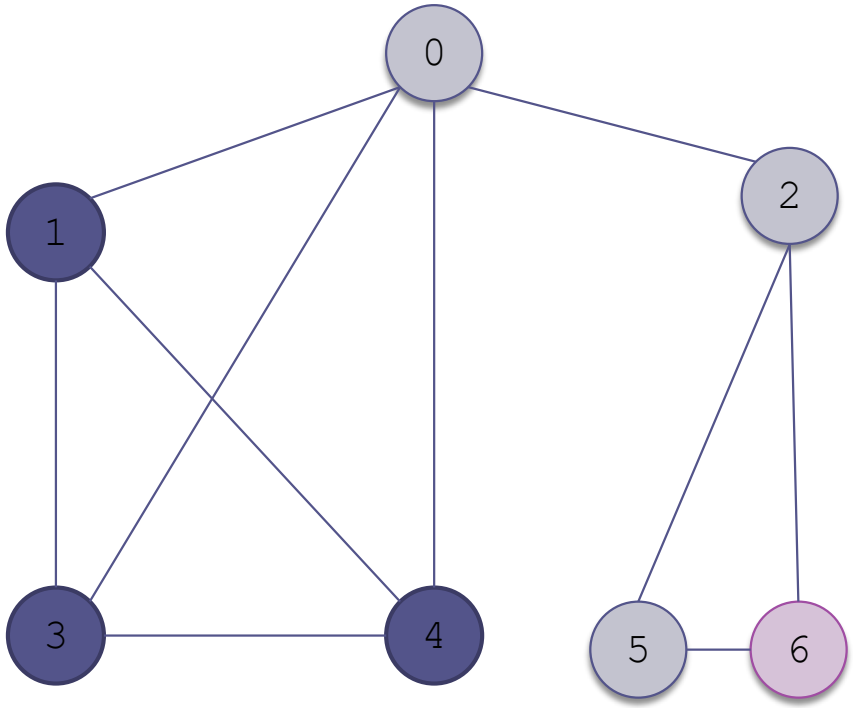
0 unvisited

0 visited

0 being visited

Example of a Depth-First Search (cont.)

6 is adjacent to 5 and is not being visited



Discovery (Visit) order:
0, 1, 3, 4, 2, 5

Finish order:
4, 3, 1

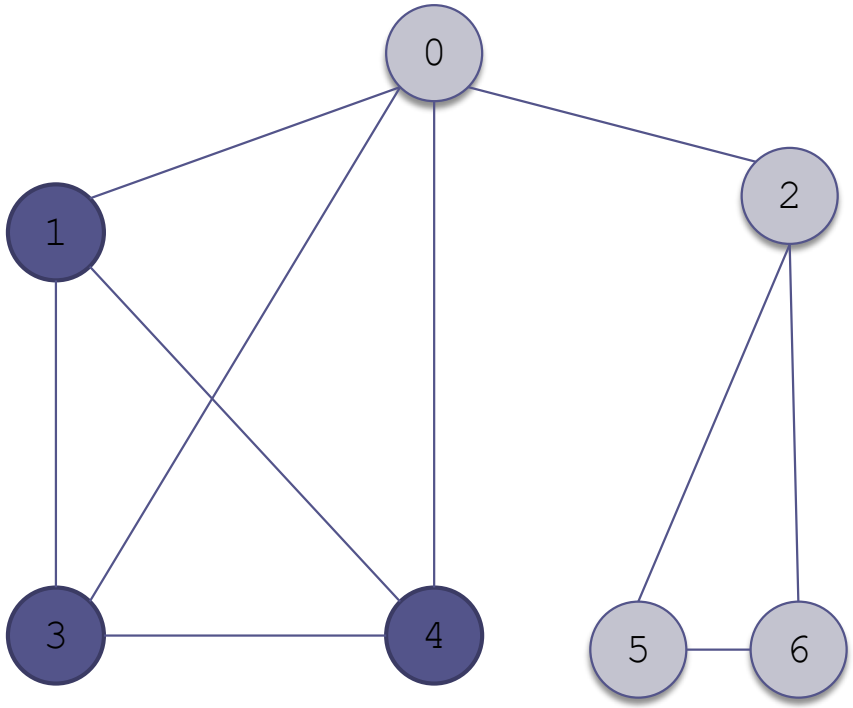
0 unvisited

0 visited

0 being visited

Example of a Depth-First Search (cont.)

6 is adjacent to 5 and is not being visited

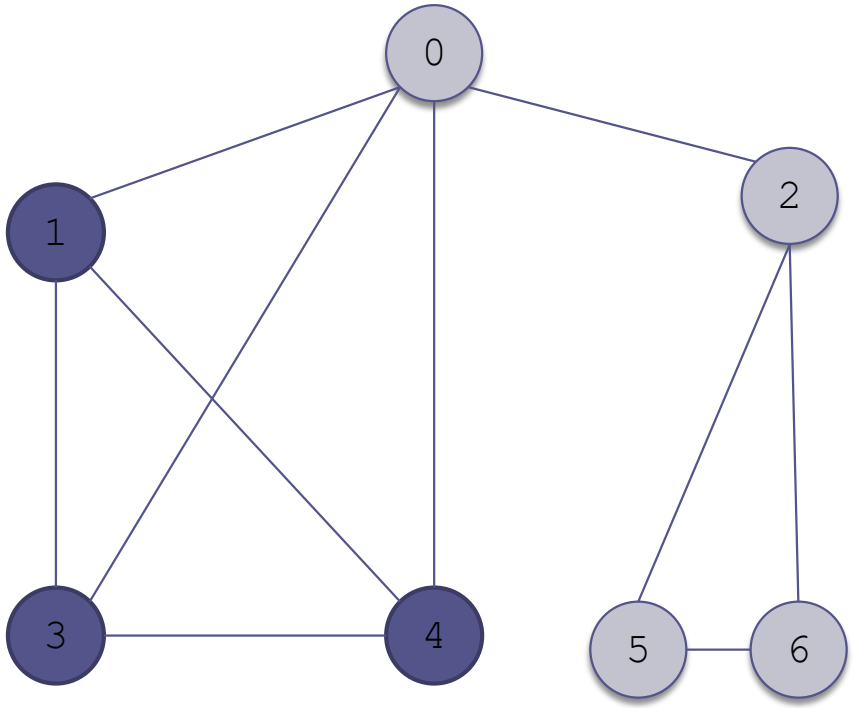


Discovery (Visit) order:
0, 1, 3, 4, 2, 5, 6

Finish order:
4, 3, 1

Example of a Depth-First Search (cont.)

There are no vertices adjacent to 6 not being visited; mark 6 as visited

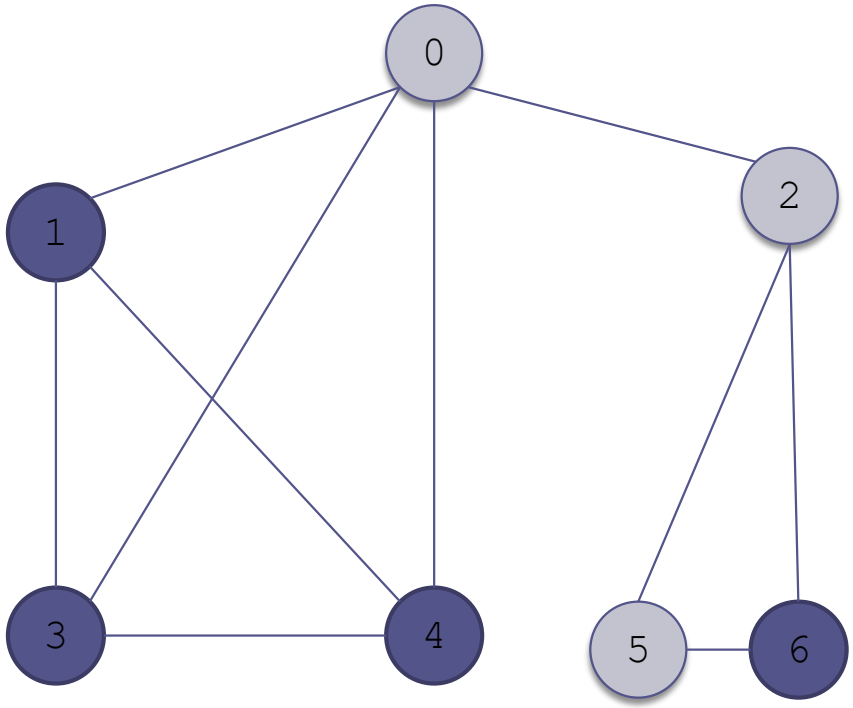


Discovery (Visit) order:
0, 1, 3, 4, 2, 5, 6

Finish order:
4, 3, 1

Example of a Depth-First Search (cont.)

There are no vertices adjacent to 6 not being visited; mark 6 as visited

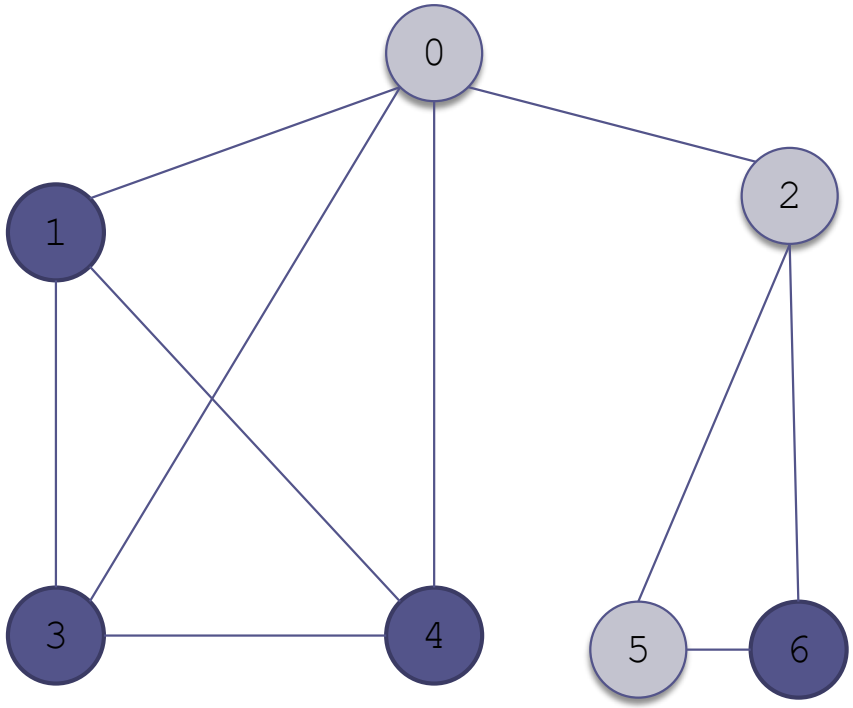


Discovery (Visit) order:
0, 1, 3, 4, 2, 5, 6

Finish order:
4, 3, 1, 6

Example of a Depth-First Search (cont.)

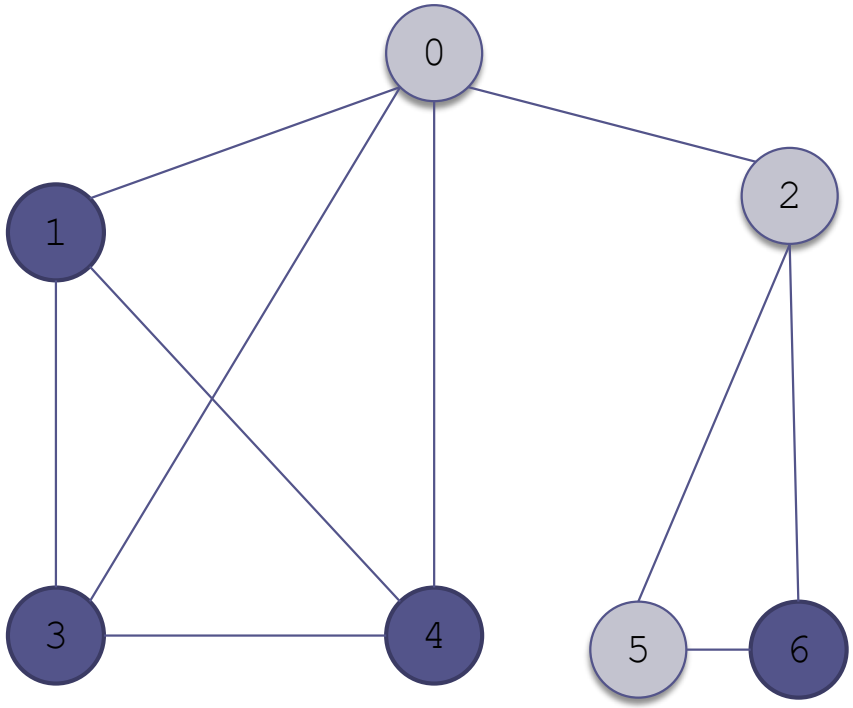
Return from the recursion to *5*



Finish order:
4, 3, 1, 6

Example of a Depth-First Search (cont.)

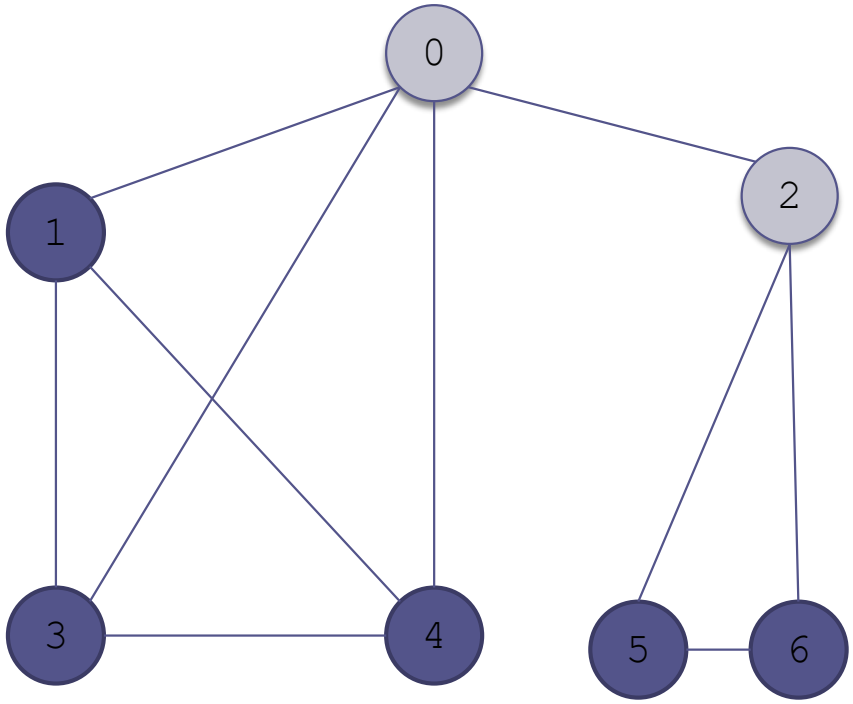
Mark 5 as visited



Finish order:
4, 3, 1, 6

Example of a Depth-First Search (cont.)

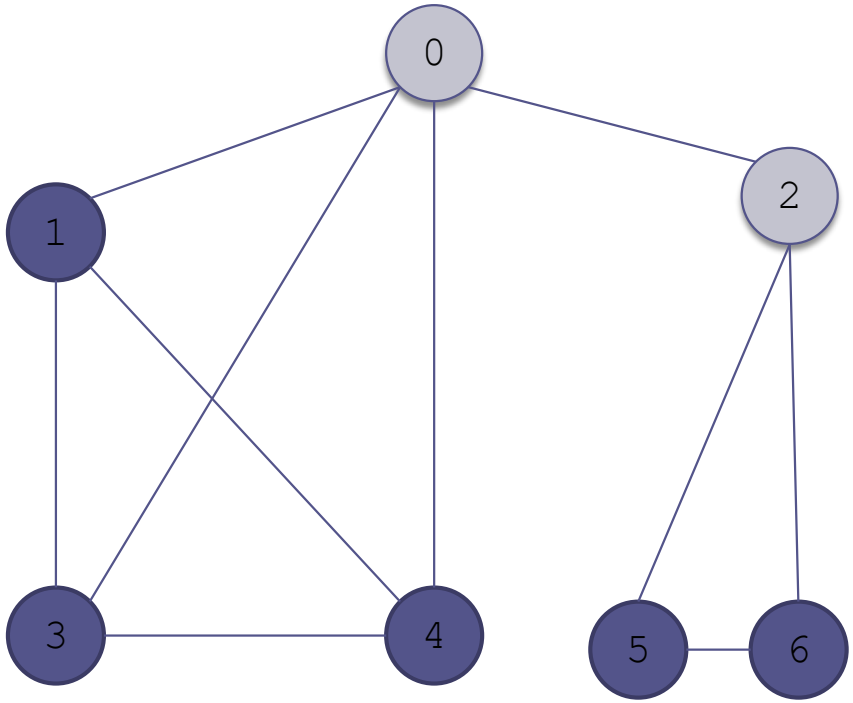
Mark 5 as visited



Finish order:
4, 3, 1, 6, 5

Example of a Depth-First Search (cont.)

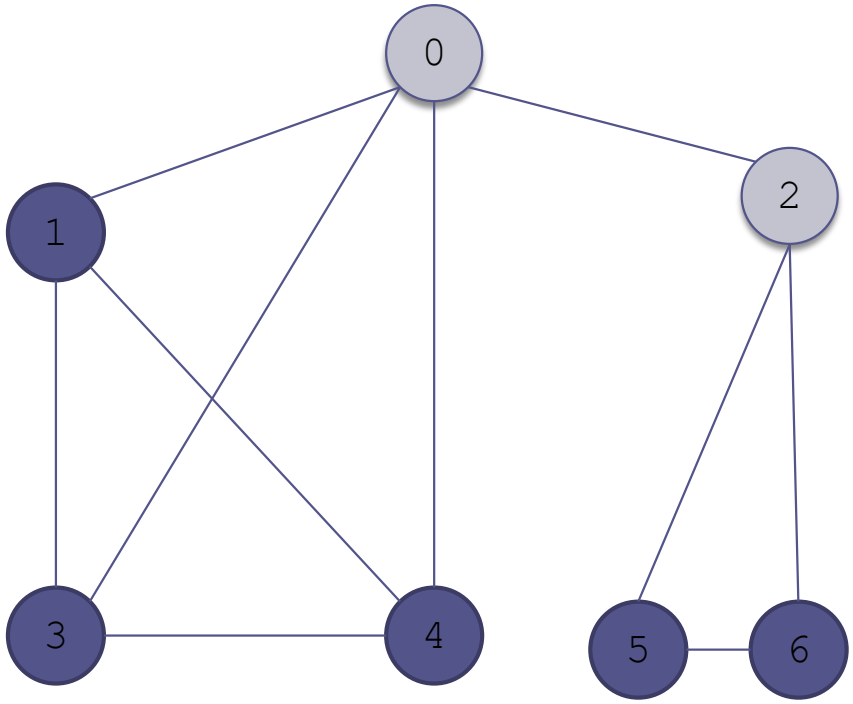
Return from the recursion to 2



Finish order:
4, 3, 1, 6, 5

Example of a Depth-First Search (cont.)

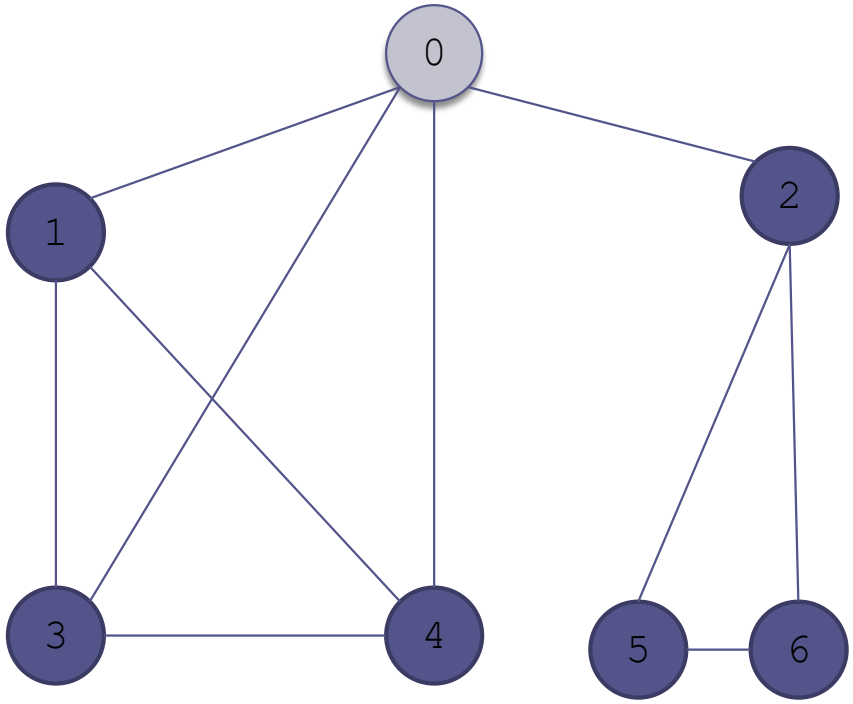
Mark 2 as visited



Finish order:
4, 3, 1, 6, 5

Example of a Depth-First Search (cont.)

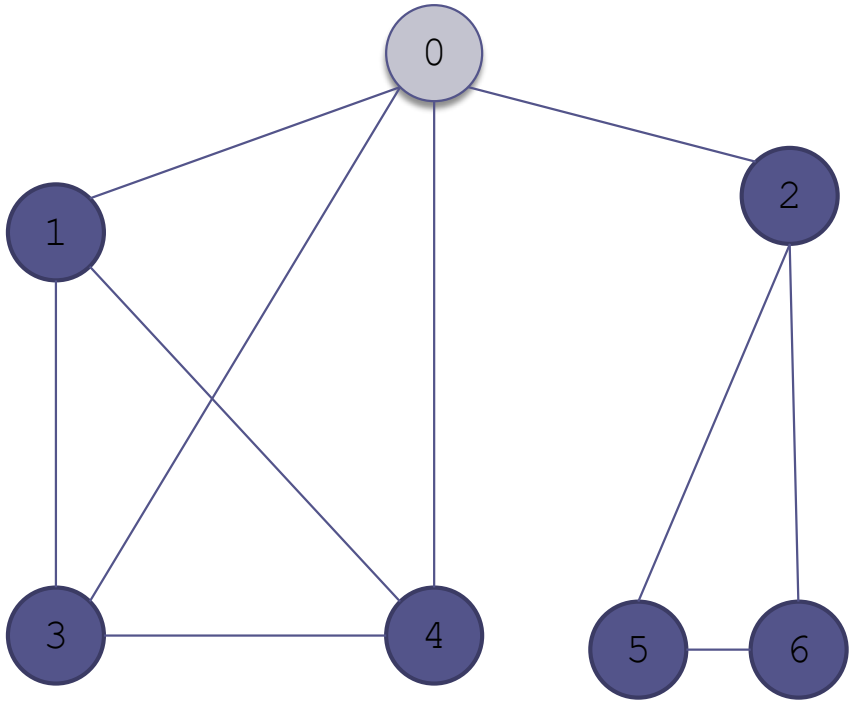
Mark 2 as visited



Finish order:
4, 3, 1, 6, 5, 2

Example of a Depth-First Search (cont.)

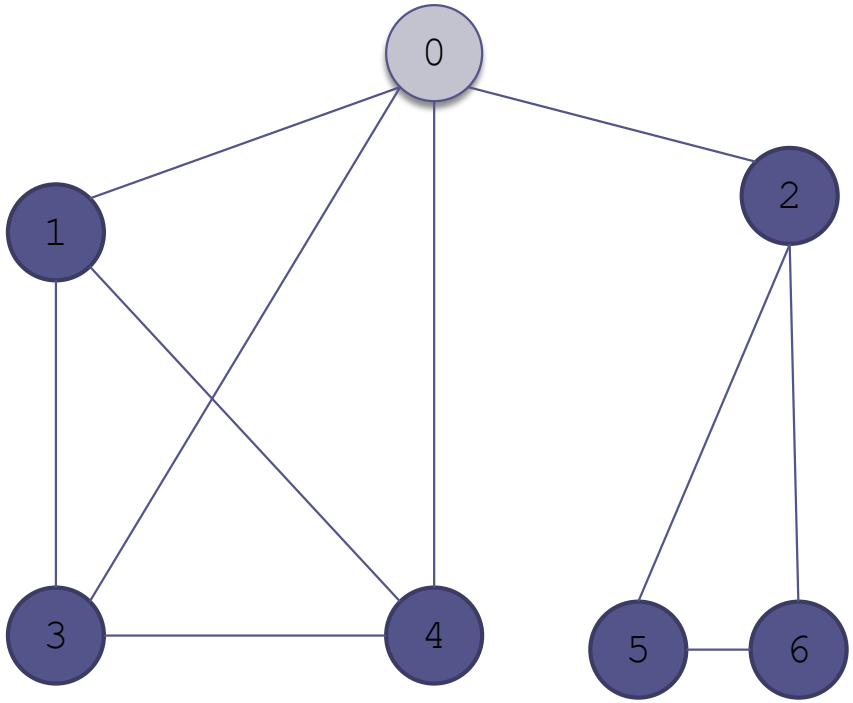
Return from the recursion to 0



Finish order:
4, 3, 1, 6, 5, 2

Example of a Depth-First Search (cont.)

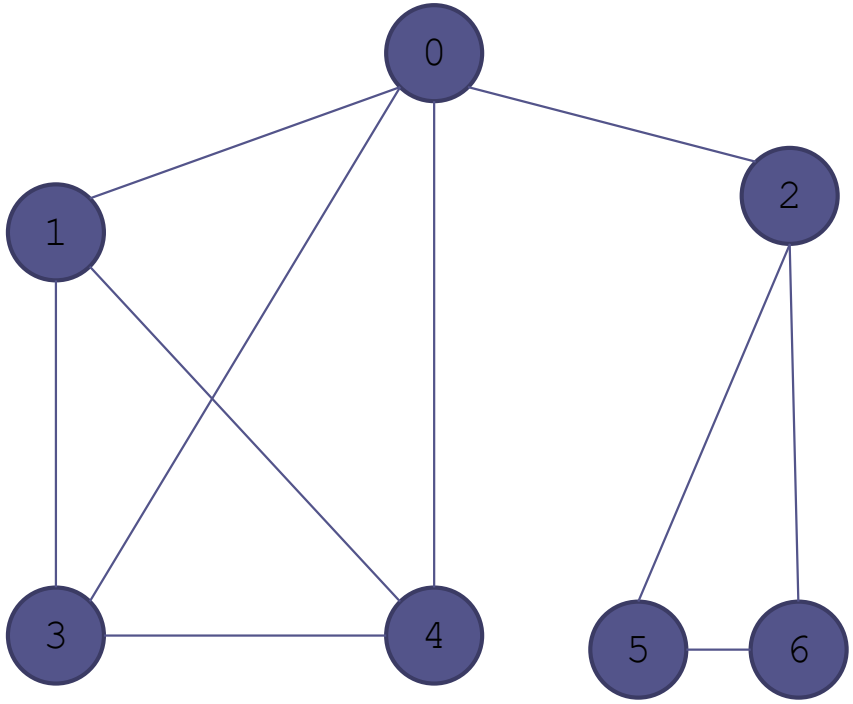
There are no nodes adjacent to 0 not being visited



Finish order:
4, 3, 1, 6, 5, 2

Example of a Depth-First Search (cont.)

Mark 0 as visited



Discovery (Visit) order:
0, 1, 3, 4, 2, 5, 6, 0

Finish order:
4, 3, 1, 6, 5, 2, 0

Algorithm for Depth-First Search

Algorithm for Depth-First Search

1. Mark the current vertex, *u*, visited (color it light blue), and enter it in the discovery order list
2. for each vertex, *v*, adjacent to the current vertex, *u*
3. if *v* has not been visited
4. Set parent of *v* to *u*.
5. Recursively apply this algorithm starting at *v*.
6. Mark *u* finished (color it dark blue) and enter *u* into the finish order list.

Performance Analysis of Depth-First Search

- The loop at Step 2 is executed $|E_v|$ times
- The recursive call results in this loop being applied to each vertex
- The total number of steps is the sum of the edges that originate at each vertex, which is the total number of edges, $|E|$
- The algorithm is $O(|E|)$

Minimum Spanning Tree

- The minimum spanning tree (MST) of a weighted connected graph is a subgraph that contains all of the nodes of the original graph and a subset of the edges so that:
  - The subgraph is connected
  - The total weights of the subgraph edges is the smallest possible

Brute Force Algorithm

- We could look at each subset of the edge set and first see if it is connected and then see if its total weight is smaller than the best answer so far

- If there are N edges in the graph, this process will require that we look at 2^N different subsets

- This is a lot more work than is needed

The Dijkstra-Prim Method

- This is a greedy algorithm that solves the larger problem by looking at a subset and making the best choice for it
- In this case, we will look at all of the edges from the starting node and choose the smallest
- On each pass, we will choose the smallest of the edges from nodes already in the MST to nodes not in the MST (the fringe)

Dijkstra-Prim Example

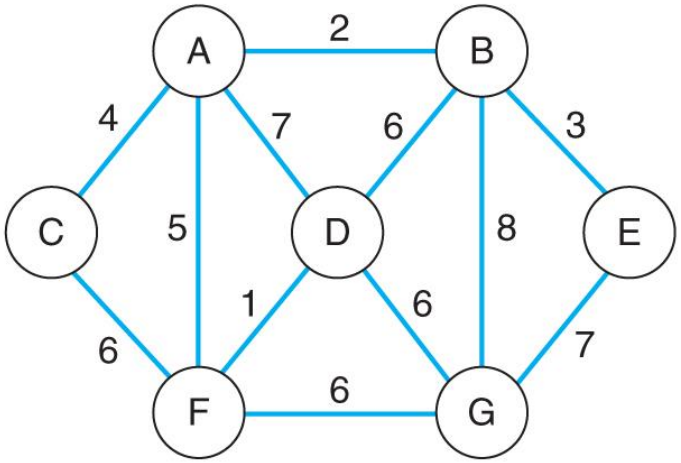


FIGURE 8.5A
The original graph

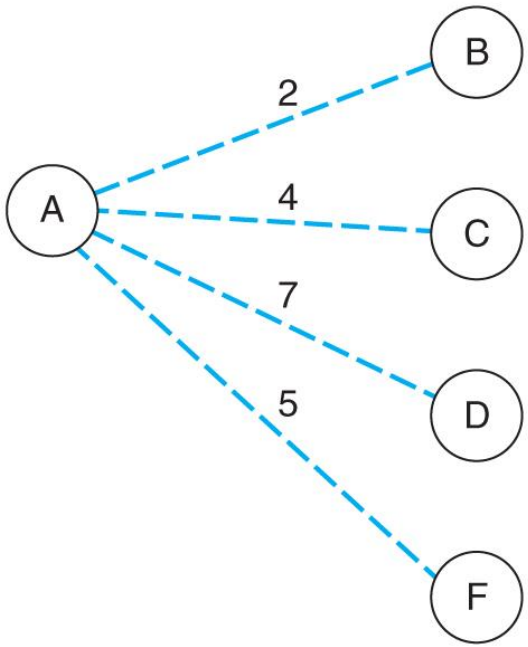


FIGURE 8.5B
First node added. (Dashed lines show edges to fringe nodes.)

Dijkstra-Prim Example

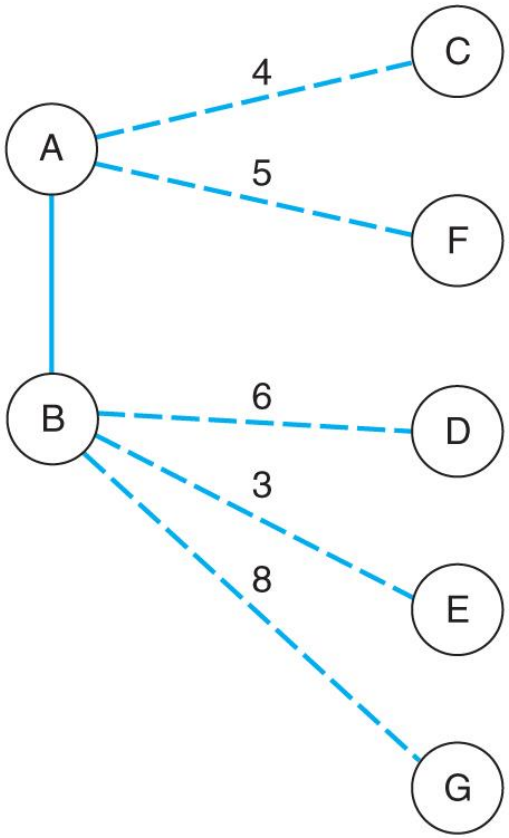


FIGURE 8.5C
Second node added. Edges to nodes D, E, and G updated. (Solid lines show edges in the tree.)

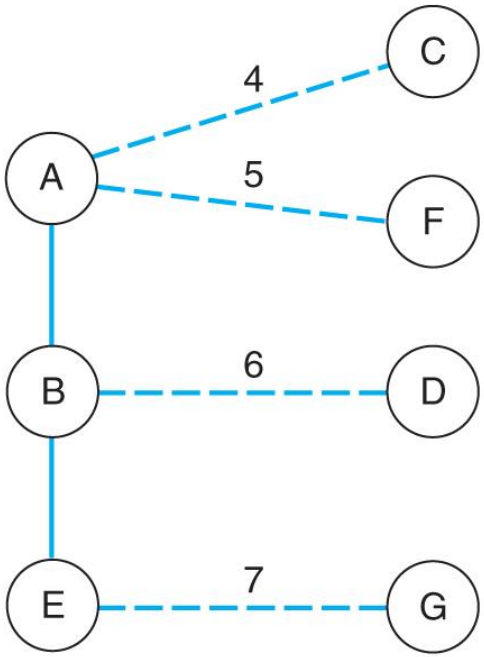


FIGURE 8.5D
Third node added. Edge to node G updated.

Dijkstra-Prim Example

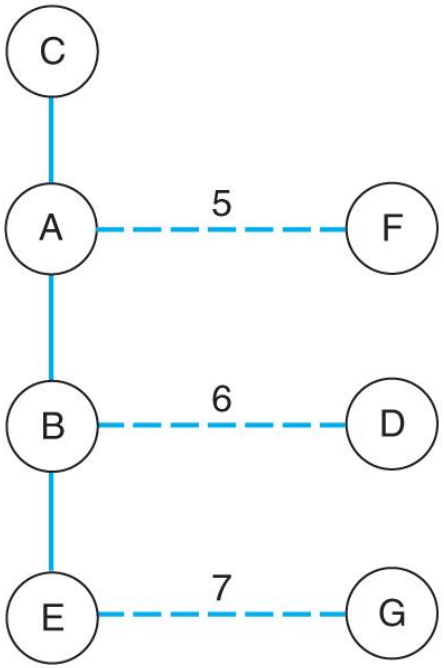


FIGURE 8.5E
Node C added to the tree

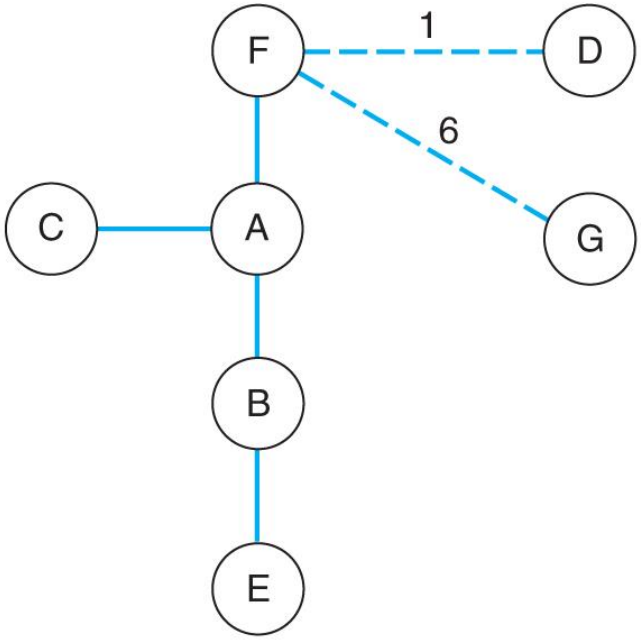


FIGURE 8.5F
Node F added to the tree. Edges to nodes D and G updated.

Dijkstra-Prim Example

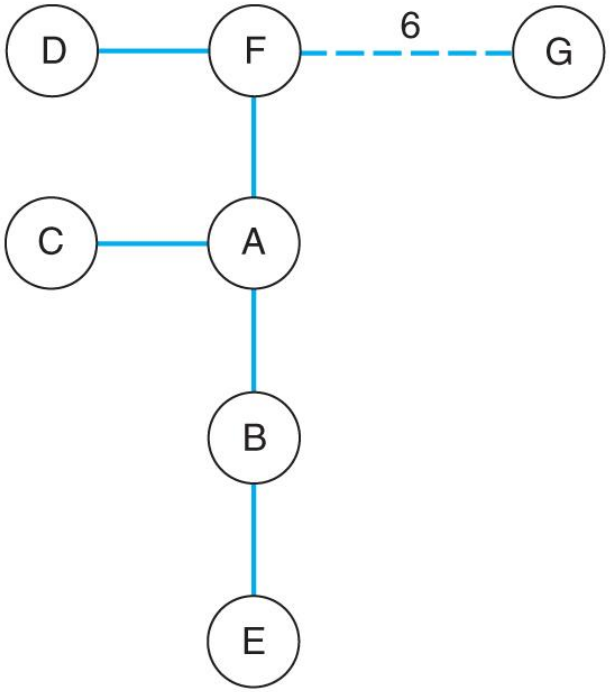


FIGURE 8.5G
Only one node is left in the fringe

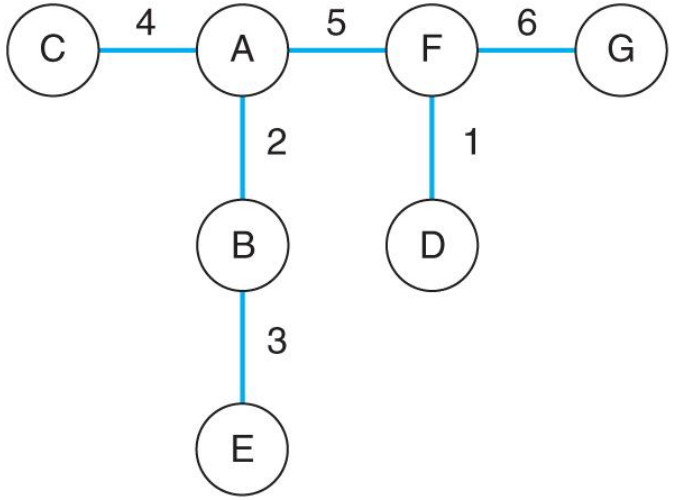


FIGURE 8.5H
The complete minimum spanning tree rooted at node A

The Dijkstra-Prim Algorithm

```
Select a starting node
build the initial fringe from nodes
      connected to the starting node
while there are nodes left do
   choose the edge to the fringe of the smallest
         weight
   add the associated node to the tree
   update the fringe by:
      adding nodes to the fringe connected to the
            new node
      updating the edges to the fringe so that
            they are the smallest
end while
```